

RTL Design Sherpa

HIVE Hardware Architecture Specification 0.2

October 13, 2026

Table of Contents

1 Hive Has Index....12

2 Document Information....13

2.1 HIVE Hardware Architecture Specification....13

2.2 Document Purpose....14

2.3 References....15

2.4 Terminology....16

2.5 Revision History....17

3 Purpose and Scope....19

3.1 Document Purpose....19

3.2 What HIVE Is....20

3.2.1 Status: specification phase, no RTL yet....20

3.3 Scope....21

3.3.1 In Scope....21

3.3.2 Explicit Non-Goals....21

3.4 Intended Audience....23

3.5 Specification Conventions....24

4 System Overview and Context....25

4.1 Executive Summary....25

4.2 Hierarchical Organization....26

4.2.1 Figure 2.1: HIVE control plane....26

4.3 Position in the RAPIDS / Delta / HIVE Ecosystem....27

4.3.1 Figure 2.2: HIVE in the RAPIDS / Delta / HIVE system....27

4.4 Control Flow Example: Matrix Multiply....29
4.5 Virtual Tile Addressing....30
4.5.1 Figure 2.3: Delta mesh and the HIVE virtual tiles....30
4.6 Packet Type Usage by Component....31
4.7 Key Architectural Features....32
5 Subsystem Architecture....33
5.1 Block 1: HIVE-C Master Controller....34
5.2 Block 2: SERV Monitors (16 instances)....35
5.3 Block 3: Control Network....36
5.4 Block 4: Configuration Manager....37
5.5 Inter-Block Communication....39
5.6 Resource Budget (NexysA7 100T)....40
6 Top-Level Interfaces....42
6.1 External Interface Summary....43
6.2 Communication Protocol Rule (owner, 2026-10-13)....44
6.3 Host Interface....45
6.4 AXIS CDA TX (HIVE-C → Delta)....46
6.5 AXIS SERV TX (16×, per-monitor injection)....47
6.6 AXIS HIVE-C RX (status return)....48
6.7 Control Network (internal)....49
6.8 Router Monitor Taps (internal)....50
7 Packet Formats and Routing Rules....51
7.1 AXIS Packet Classification....52
7.2 CDA Descriptor Format (256-bit)....53

7.3 PKT_CONFIG Format (128-bit)..... 54
7.4 PKT_STATUS Format (128-bit)..... 55
7.5 TID Enforcement Rule..... 56
8 Memory Map and Register Spaces..... 57
8.1 HIVE-C Firmware Memory Map..... 58
8.2 Router Configuration Registers (per tile)..... 59
8.3 Soft Reset Register..... 60
8.4 Fabric IDLE Status Register..... 61
8.5 Register Access Rules..... 62
9 HIVE-C Firmware Architecture..... 63
9.1 Main Loop..... 64
9.2 Descriptor Injection API..... 65
9.3 Interrupts..... 66
10 Descriptor Scheduling..... 67
10.1 Priority Levels..... 68
10.2 Scheduling Policy..... 69
10.3 Channel-Aware S2MM Programming..... 70
10.4 Worked Example: Matrix-Multiply Layer..... 71
11 SERV Monitor Programming..... 72
11.1 Trigger Conditions..... 73
11.2 Inband Packet Injection..... 74
11.3 Reporting Discipline..... 75
12 Network Reconfiguration Workflow..... 76
12.1 Atomic Switch Protocol..... 77

12.2 Worked Flow: Mesh to Tree Reduction....78
12.3 Safety Rules....79
13 Performance Characteristics....80
13.1 Latency Targets....81
13.2 Throughput and Overhead Targets....82
13.3 Design Tradeoffs....83
13.4 Success Criteria....84
14 Integration and Verification....85
14.1 RAPIDS Integration....85
14.2 Delta Integration....86
14.2.1 Figure 7.1: HIVE/Delta tile seam — SERV monitor above the Delta-owned NI....86
14.3 System Top Integration....87
14.4 Verification Strategy....88
14.5 Implementation Roadmap....90
14.6 Expansion Path (PoC → Production)....91
15 HIVE Micro-Architecture Specification Index....92
15.1 Document Organization....93
15.1.1 Front Matter....93
15.1.2 Chapter 1: Overview....93
15.1.3 Chapter 2: Functional Blocks....93
15.1.4 Chapter 3: Internal Interfaces....93
15.1.5 Chapter 4: Micro-Architecture Behavior....93
15.1.6 Chapter 5: Verification Hooks....93

15.1.7 Chapter 6: Configuration....93
15.2 Related Documentation....94
16 Product Requirements Document (PRD)....95
16.1 HIVE - Hierarchical Intelligent Vector Environment....95
16.2 1. Executive Summary....96
16.2.1 1.1 Quick Stats....96
16.2.2 1.2 Subsystem Goals....96
16.3 2. Documentation Structure....97
16.3.1 HIVE Hardware Architecture Specification (HAS) — external contract....97
16.3.2 HIVE Micro-Architecture Specification (MAS) — implementation intent....97
16.3.3 Other Documentation....97
16.4 3. Architecture Overview....98
16.4.1 3.1 Top-Level Block Diagram....98
16.4.2 3.2 Resource Budget (NexysA7 100T)....98
16.5 4. Key Features....100
16.5.1 4.1 Hierarchical RISC-V Architecture....100
16.5.2 4.2 Distributed Monitoring....100
16.5.3 4.3 Network Reconfiguration....100
16.5.4 4.4 RAPIDS Integration....101
16.6 5. Interfaces....102
16.6.1 5.1 External Interfaces....102
16.6.2 5.2 Virtual Tile Mapping....102

16.7 6. Use Cases....103
16.7.1 6.1 Descriptor Scheduling....103
16.7.2 6.2 Congestion Management....103
16.7.3 6.3 Network Reconfiguration....103
16.8 7. Test Coverage....104
16.8.1 7.1 Current Status....104
16.9 8. Performance Targets....105
16.9.1 8.1 Latency Targets....105
16.9.2 8.2 Monitoring Overhead....105
16.10 9. Integration Guidelines....106
16.10.1 9.1 HIVE + RAPIDS + Delta Integration....106
16.11 10. Development Status....108
16.11.1 10.1 Current Phase....108
16.11.2 10.2 Roadmap....108
16.11.3 10.3 HIVE-C core strategy — VexRiscv now, falcon-flavored later…108
16.12 11. Educational Value....110
16.13 13. Documentation Generation....111
16.13.1 13.1 Generating PDF/DOCX from the Specifications....111
16.14 14. References....112
16.14.1 14.1 Internal Documentation....112
16.14.2 14.2 Related Subsystems....112
16.14.3 14.3 External References....112
16.15 Navigation....113
17 Claude Code Guide: HIVE Component....114

17.1 Quick Context....115
17.2 Critical Rules for This Component....116
17.2.1 Rule #2: Always Reference the Specifications....116
17.2.2 Rule #3: HIVE System Context....116
17.3 Architecture Quick Reference....117
17.3.1 Block Organization (HIVE System)....117
17.4 Common User Questions and Responses....118
17.4.1 Q: “What’s the difference between HIVE-C and SERV monitors?”....118
17.4.2 Q: “How does HIVE-C send descriptors to RAPIDS?”....118
17.4.3 Q: “How does network reconfiguration work?”....119
17.4.4 Q: “What do SERV monitors track?”....119
17.5 Key Documentation Links....120
17.5.1 Primary Technical Specifications....120
17.5.2 Related Component Specifications....120
17.6 Documentation Generation....121
17.6.1 Generating PDF/DOCX from the Specifications....121
17.7 Remember....122
18 RAPIDS Beats Architecture HAS....123
18.1 Document Organization....124
18.1.1 Front Matter....124
18.1.2 Chapter 1: Overview....124
18.1.3 Chapter 2: Architecture....124
18.1.4 Chapter 3: Interfaces....124
18.1.5 Chapter 4: Use Cases....124

18.1.6 Chapter 5: Programming Model....124
18.1.7 Chapter 6: Performance....125
18.2 Quick Reference....126
18.2.1 RAPIDS Beats at a Glance....126
18.2.2 Interface Summary....126
18.3 Related Documentation....127
18.4 Specification Conventions....128
18.4.1 Signal Naming....128
18.4.2 Diagram Conventions....128
19 Delta Hardware Architecture Specification Index....129
19.1 Overview....129
19.2 Related Modules....130
19.3 Navigation....131
19.3.1 Front Matter....131
19.3.2 Chapter 1: Introduction....131
19.3.3 Chapter 2: System Overview....131
19.3.4 Chapter 3: Architecture....131
19.3.5 Chapter 4: Interfaces....131
19.3.6 Chapter 5: Performance....131
19.3.7 Chapter 6: Integration....131
19.3.8 Chapter 7: Mesh Network....132
20 RISC-V Falcon Suite....133
20.1 The ladder....134
20.1.1 Moved: hive-serv....135

20.1.2 Tetrad engine: gatling-rv32itetrad.....135
20.2 Goals per rung.....136
20.3 Packaging (per rung, the MC-line treatment).....137
20.4 References and layout.....138

List of Tables

Table 1: Reference Documents....15
Table 2: Terminology and Definitions....16
Table 3: Document Revision History....17
Table 4: Explicit non-goals of HIVE....21
Table 5: Intended audience....23
Table 6: Packet type usage by component....30
Table 7: HIVE-C key specifications....33
Table 8: SERV monitor key specifications....34
Table 9: Control network key specifications....35
Table 10: Configuration Manager key specifications....36
Table 11: Virtual configuration contexts....36
Table 12: Inter-block communication summary....38
Table 13: Resource budget, NexysA7 100T....39
Table 14: External interface summary....42
Table 15: Firmware module responsibilities....62
Table 16: HIVE-C interrupt inventory....65
Table 17: Descriptor priority classes....67
Table 18: SERV trigger conditions....72
Table 19: Latency targets....80
Table 20: Throughput and overhead targets....81
Table 21: VC tradeoff (per router estimates)....82
Table 22: SERV interval tradeoff....82
Table 23: Formal property inventory....87
Table 24: Implementation roadmap....89
Table 25: RAPIDS Beats Feature Summary....124
Table 26: External Interface Summary....124

1 Hive Has Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 HIVE Hardware Architecture Specification

Document Number: HIVE-HAS-001 **Version:** 0.4 **Status:** Draft (pre-RTL — specification phase)
Classification: Open Source - MIT License

2.2 Document Purpose

This Hardware Architecture Specification (HAS) defines the external, system-level architecture of the HIVE subsystem (Hierarchical Intelligent Vector Environment): what it is, what it controls, how it connects to the host, the RAPIDS DMA engine, and the Delta Network, how packets and registers are encoded at its boundaries, and how it is integrated into a larger system. It is written for consumers of the subsystem — integrators, firmware engineers, and verification engineers — and is deliberately interface-first: ports, packet formats, encodings, and protocols, with rationale where boundary behavior is a design decision rather than an obvious default.

This document does not describe the internal microarchitecture. Block internals, cycle-level behavior, clock/reset implementation detail, and the verification hook plan live in the companion Micro-Architecture Specification (MAS).

Target Audience: - System architects evaluating HIVE for integration alongside RAPIDS and Delta - Hardware engineers wiring HIVE into a system top level - Firmware engineers writing HIVE-C control software or SERV monitor firmware - Verification engineers planning subsystem-level and system-level test coverage

Companion Documents: - HIVE Micro-Architecture Specification (MAS) - internal block-level implementation intent - HIVE PRD (`PRD.md`) - product requirements, use cases, and roadmap - RAPIDS Beats HAS/MAS - the DMA engine HIVE-C controls - Delta HAS/MAS - the network HIVE-C and the SERV monitors inject packets into

---

2.3 References

Reference Documents

ID	Document	Description
[REF-1]	HIVE MAS v0.2 (`docs/hive_mas/`)	Micro-Architecture Specification (companion)
[REF-2]	HIVE PRD (`PRD.md`)	Product requirements and use cases
[REF-3]	RAPIDS Beats HAS (`../../../../../dma-ip/rapids/docs/rapids_beats_has/rapids_beats_has_index.md`)	RAPIDS DMA engine; CDA consumer
[REF-4]	Delta HAS (`../../../../../fabric-gen-ip/delta/docs/delta_has/delta_has_index.md`)	4×4 mesh NoC; packet routing fabric
[REF-5]	AXI4-Stream Protocol Specification, ARM IHI 0051A	AXIS signal semantics (TUSER/TID/TDEST/TLAST)
[REF-6]	VexRiscv (https://github.com/SpinalHDL/VexRiscv)	HIVE-C master core generator
[REF-7]	SERV (https://github.com/olofk/serv)	Bit-serial RV32I core used for the 16 monitors
[REF-8]	RISC-V Falcon Suite README (`../README.md`)	Suite ladder and hive-serv placement beside it

2.4 Terminology

Terminology and Definitions

Term	Definition
CDA	Compute Direct Access — the packet class (TUSER=2'b01) HIVE-C uses to inject RAPIDS descriptors inband
Context	One of 4 pre-loaded routing configurations (systolic, mesh, tree, custom) held by the Configuration Manager
Control Network	Dedicated low-bandwidth star network between HIVE-C and the 16 SERV monitors, separate from the Delta data path
HIVE-C	The HIVE master controller: one VexRiscv RV32IM core, Delta Virtual Tile 17
Inband Control	Control messages carried through the data network as packets rather than over a separate memory-mapped bus
Monitors	The 16 SERV-based per-tile traffic monitors (not to be confused with verification monitors)
PKT_DATA / PKT_CONFIG / PKT_STATUS	AXIS packet classes encoded in TUSER[1:0]: 2'b00 compute data, 2'b10 configuration, 2'b11 status
Quiesce	The drain-and-hold protocol that empties the network before an atomic context switch
SERV	Small Embedded RISC-V — bit-serial RV32I core, ~125 LUTs per instance
Virtual Tile	A Delta Network endpoint that is not a physical compute tile: 16 = RAPIDS, 17 = HIVE-C
VexRiscv	SpinalHDL-generated RISC-V core family; HIVE-C uses the "Small" RV32IM configuration

2.5 Revision History

Document Revision History

Version	Date	Author	Description
0.1	2026-10-12	seang	Initial HAS release. Migrates and supersedes the v0.3 `hive_spec/` + `hive_specification.md` drafts; resource-budget table corrected (HIVE-only vs. system subtotals disambiguated); delta references repointed to `fabric-gen-ip/delta`
0.2	2026-10-13	seang	Owner decisions recorded: fabric IDLE status register (PeakRDL bank) is the quiesce check; kestrel loader pattern for firmware load; descriptor mailbox parameter-controlled with dedicated register file default (custom-instruction alternative rejected). Addendum: delta relationship closed — the AXIS fabric generator is the fabric (same fabric as the archived NoC draft); AXIS-only communication rule — all block-boundary traffic is AXI4-Stream, control network AXIS-shaped, AXI4-Lite for registers only, no AXI4 full
0.3	2026-10-20	seang	Block diagrams redrawn to

Version	Date	Author	Description
			the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.
0.4	2026-10-20	seang	Section (h2) headings now open a new page, the pipeline default. No technical content changed.

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 Purpose and Scope

3.1 Document Purpose

This Hardware Architecture Specification is the integration contract for the HIVE subsystem. It specifies:

1. **System architecture** - the hierarchical control plane (one VexRiscv master, sixteen SERV monitors) and its position in the RAPIDS/Delta ecosystem
2. **Subsystem architecture** - the four functional blocks (HIVE-C, SERV monitors, Control Network, Configuration Manager), their responsibilities, and the resource budget on the proof-of-concept FPGA
3. **External interfaces** - top-level interface classes, the AXIS packet type encoding, the 256-bit CDA descriptor format, PKT_CONFIG/PKT_STATUS formats, and the memory-mapped register spaces
4. **Programming model** - HIVE-C firmware architecture, descriptor scheduling, SERV monitor programming, and the network reconfiguration workflow
5. **Performance targets** - latency, throughput, overhead, and monitoring budget commitments, plus the tradeoff tables those commitments come from
6. **Integration and verification** - RAPIDS/Delta integration recipes, test levels, formal property inventory, and the implementation roadmap

3.2 What HIVE Is

HIVE (Hierarchical Intelligent Vector Environment) is a distributed control and monitoring subsystem for the RAPIDS DMA engine and the Delta Network compute fabric. One VexRiscv master controller (HIVE-C) coordinates global operations; sixteen SERV bit-serial cores, one per Delta compute tile, provide distributed traffic monitoring and inband configuration/status injection. HIVE consumes no DSP blocks and a minority of FPGA logic, leaving the fabric free for compute.

HIVE lives in `projects/components/compute-eng-ip/hive-serv/`, beside the RISC-V falcon suite ladder. It is deliberately **not** a falcon rung: the ladder teaches one core's microarchitecture per rung, while HIVE *uses* RISC-V cores (VexRiscv and SERV, both external projects) as building blocks for a control system. The educational payload here is hierarchical control, distributed monitoring, and inband control packets — not a new CPU.

3.2.1 Status: specification phase, no RTL yet

This HAS describes a planned subsystem. No RTL, firmware, or testbenches exist yet; the companion MAS documents implementation intent to seed that work. Where a number in this document is a target rather than a measured result, it is said so. Content migrated here from the earlier v0.3 spec drafts is treated as design intent and remains subject to change during RTL bring-up.

3.3 Scope

3.3.1 In Scope

- The four HIVE blocks and their responsibilities (HIVE-C, 16× SERV monitors, Control Network, Configuration Manager)
- AXIS packet classification (TUSER[1:0]) and routing rules on the Delta Network
- The 256-bit CDA descriptor format and HIVE-C → RAPIDS injection contract
- PKT_CONFIG and PKT_STATUS formats, including the SERV → HIVE-C reporting path
- Network reconfiguration: the 4 virtual contexts and the atomic switch protocol
- HIVE-C firmware architecture, descriptor scheduling policy, and memory map
- Resource budget on the Digilent NexysA7 100T proof-of-concept platform
- Performance targets and the tradeoff tables behind them
- Integration recipes against RAPIDS and Delta, and the verification strategy

3.3.2 Explicit Non-Goals

The following are **not** provided by HIVE, and integrators must not rely on them:

Explicit non-goals of HIVE

Non-goal	What provides it instead
Compute elements (DSP MACs, PE arrays)	Delta Network compute tiles; HIVE only monitors and configures them
DDR memory controller	RAPIDS' AXI4 memory interface; HIVE never touches DDR directly
Data-plane packet switching / routing tables	Delta Network routers; HIVE *writes* their configuration, it does not forward packets
General-purpose host OS / driver stack	Bare-metal host commands over UART or AXI4-Lite; no OS driver exists
Real-time guarantees (hard	Best-effort latency targets; deadline-aware descriptor scheduling is a policy, not a guarantee

Non-goal	What provides it instead
deadlines)	
Cache coherence	None in the PoC; coherent-access flag in the descriptor is reserved for future use
RTL, firmware binaries, SimPy models	Planned deliverables; this document is the specification they will be verified against

3.4 Intended Audience

Intended audience

Audience	Primary use of this document
System architects	Decide whether HIVE fits the control plane; understand the contracts with RAPIDS and Delta
Hardware engineers	Wire `hive_top` into a system; close timing against the interface classes; plan clock domains
Firmware engineers	Structure HIVE-C control software; write SERV monitor firmware; link images for both cores
Verification engineers	Plan subsystem-level coverage against Chapter 7 and the MAS verification hooks

3.5 Specification Conventions

- Signal, register, and packet-field names appear in code font exactly as used in the RTL plan (`axis_cda_tvalid`, `MODE_SELECT`) so every claim can be grepped once RTL lands.
- Hexadecimal values use `0x` or SystemVerilog sizing (`32'hFFFF_FFFE`); packet type encodings use sized binary (`2'b01`).
- "Cycle" means one cycle of the relevant clock domain (see MAS clocking chapter for the four-domain map); cross-domain latencies are given as approximate ranges.
- **Monitor** in this book means an SERV traffic monitor (a HIVE block). The repository's verification monitor system (Ocelus) is referred to by name to avoid the collision.
- Packet type names follow the Delta convention: `PKT_DATA`, `CDA` (called `PKT_DESC` in early drafts), `PKT_CONFIG`, `PKT_STATUS` in TUSER[1:0].

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 System Overview and Context

4.1 Executive Summary

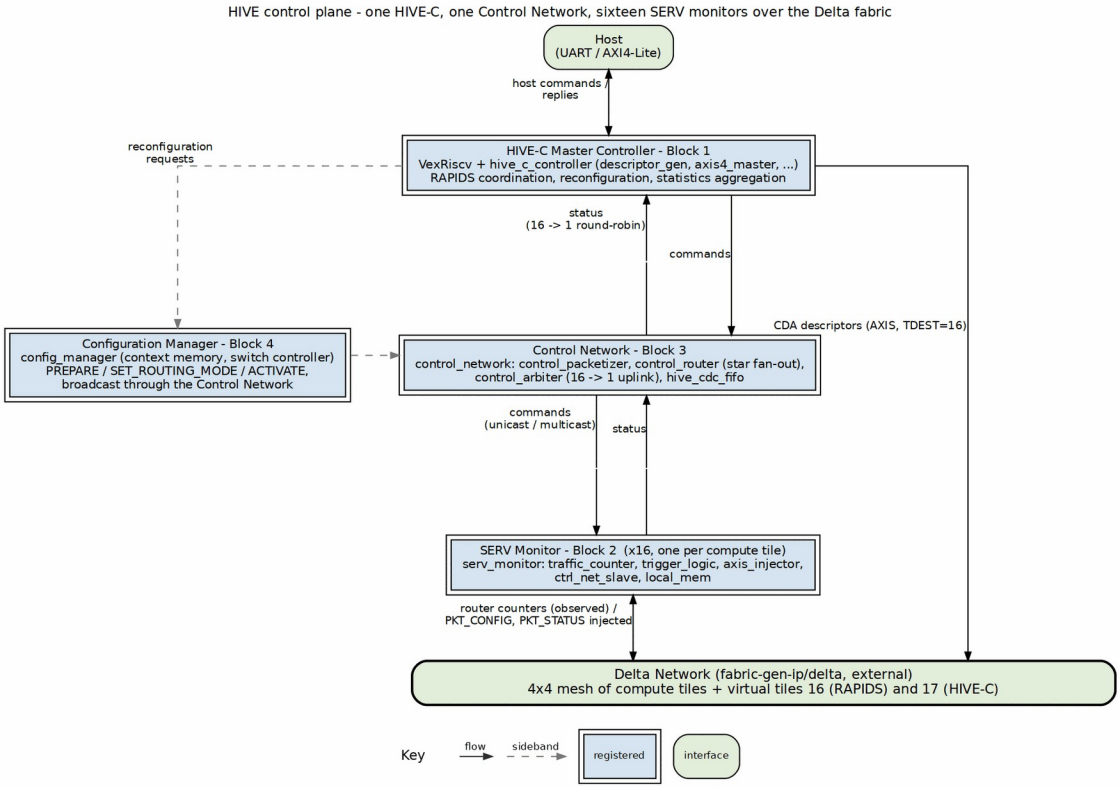
HIVE is a distributed control and monitoring subsystem coordinating the RAPIDS DMA engine and the Delta Network compute fabric. A hierarchical RISC-V architecture provides it: one VexRiscv master controller manages global operations, and sixteen lightweight SERV cores — one per Delta compute tile — provide per-tile traffic monitoring and inband descriptor/configuration injection.

Primary design goals: - Low-latency, inband control of RAPIDS DMA through AXIS-native descriptor injection (~10-20 cycles vs. 100+ for memory-mapped polling) - Distributed traffic monitoring across the Delta compute network - Reconfigurable network topologies through virtual configuration switching - A comprehensive educational platform with performance models and modular design

Target platforms: - **Proof of Concept:** Digilent NexysA7 100T (Xilinx Artix-7 XC7A100T) - **Production target:** Digilent Genesys2 (Xilinx Kintex-7 325T) - **Alternative:** Terasic DE10 Standard (Cyclone V SX, Quartus toolchain)

4.2 Hierarchical Organization

4.2.1 Figure 2.1: HIVE control plane



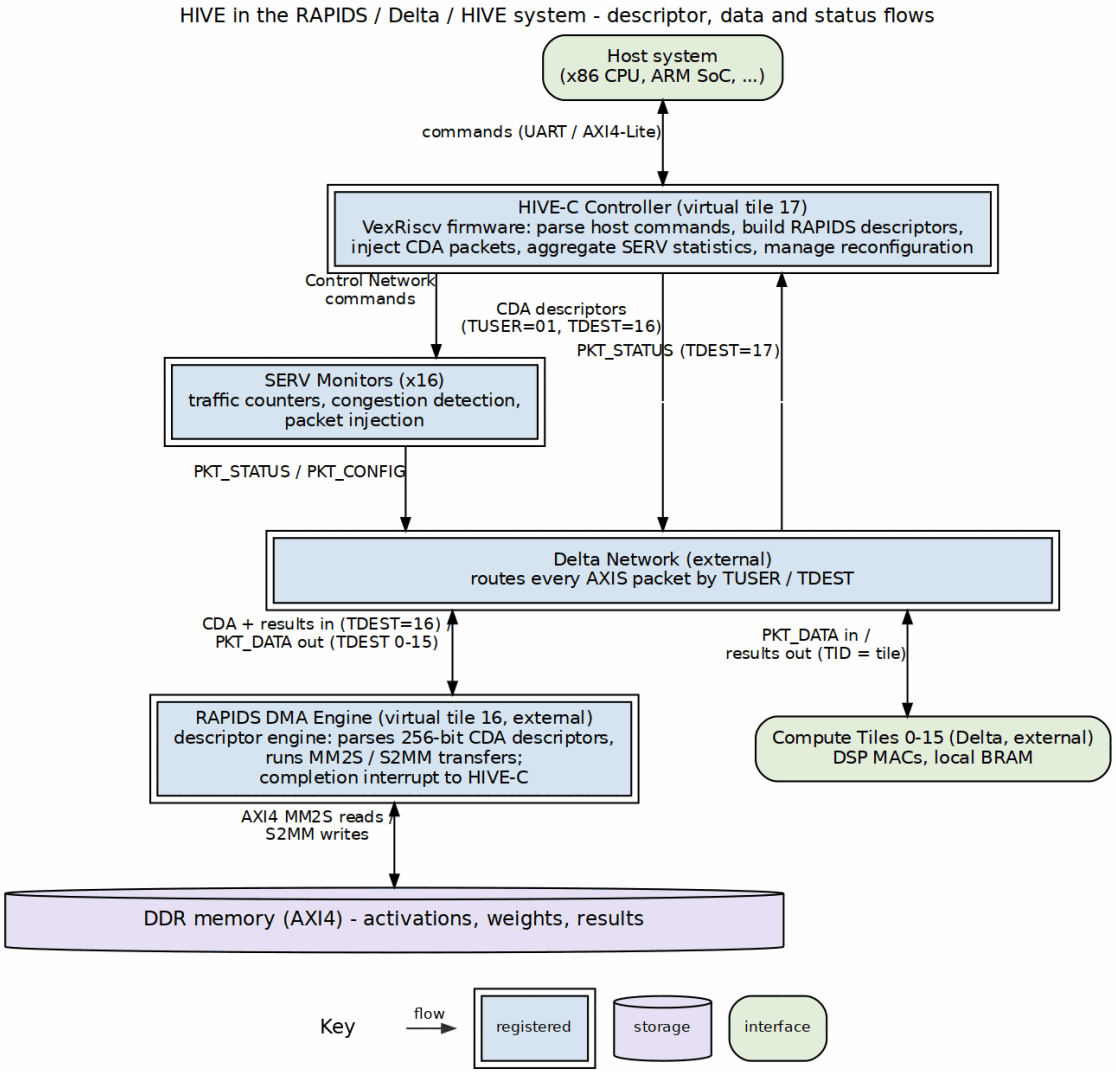
HIVE control plane: HIVE-C, Control Network, sixteen SERV monitors, over the Delta network

The compute tiles, routers, and network interfaces belong to Delta (see the Delta HAS). HIVE owns the control plane above them: HIVE-C, the sixteen SERV monitors, the Control Network, and the Configuration Manager.

4.3 Position in the RAPIDS / Delta / HIVE Ecosystem

Delta relationship (owner, 2026-10-13): the delta component (`projects/components/fabric-gen-ip/delta/`) — the AXIS fabric generator — *is* the fabric HIVE communicates with. The archived NoC-mesh draft (`noc-ip/delta_spec/`) and the generator describe the same fabric; the 2026-10-12 re-home from `noc-ip/` was organizational. HIVE's architecture stands as specified against delta, and HIVE talks to it exclusively over AXI4-Stream (§4.1).

4.3.1 Figure 2.2: HIVE in the RAPIDS / Delta / HIVE system



System data flow: host, HIVE-C, Delta network, RAPIDS, compute tiles, DDR

4.4 Control Flow Example: Matrix Multiply

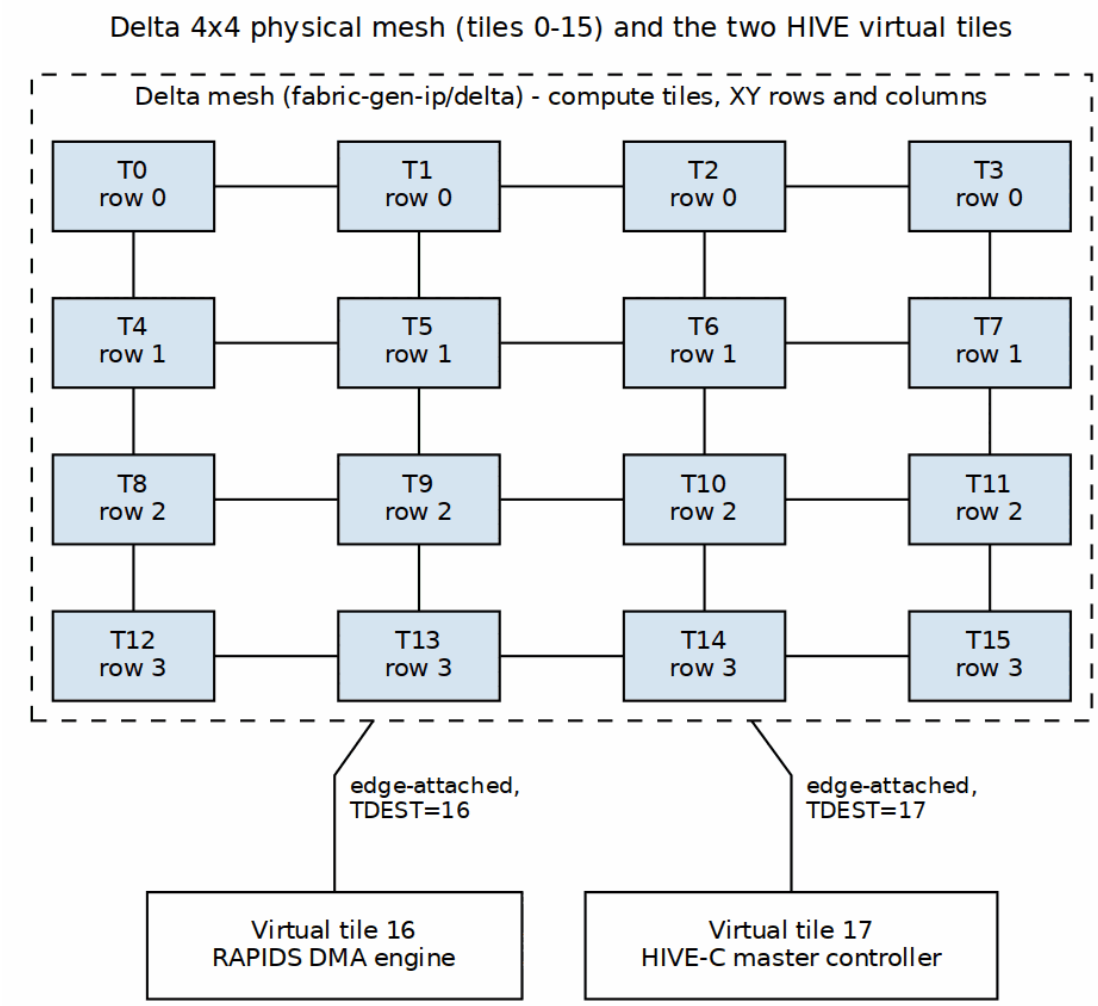
The end-to-end flow that exercises every HIVE block:

1. **Setup (HIVE-C):** Host sends “matmul layer N, tiles 0-7”. HIVE-C switches Delta to mesh mode if needed, then generates MM2S descriptors (DDR → tiles) and S2MM descriptors (tiles → DDR).
2. **Descriptor injection (HIVE-C → RAPIDS):** 256-bit descriptors are sent as CDA packets (TUSER=2’b01, TDEST=16); Delta routes them to the RAPIDS descriptor engine.
3. **Data movement (RAPIDS → tiles):** RAPIDS executes MM2S descriptors, reads activations from DDR over AXI4, and emits PKT_DATA (TUSER=2’b00, TDEST=0-7).
4. **Computation (tiles):** Compute elements produce results and send PKT_DATA with TID = source tile. The tile Network Interface overrides TID with the local tile ID (hardware-enforced, see §4.2).
5. **Result collection (tiles → RAPIDS → DDR):** PKT_DATA routed to RAPIDS (TDEST=16); S2MM channels match on TID; results written to DDR; completion interrupt to HIVE-C.
6. **Monitoring (SERV → HIVE-C):** SERV monitors track packet counts and buffer occupancy, alert on congestion, and send PKT_STATUS to HIVE-C (TDEST=17).

4.5 Virtual Tile Addressing

HIVE participates in the Delta topology as two virtual tiles:

4.5.1 Figure 2.3: Delta mesh and the HIVE virtual tiles



Delta 4x4 mesh of compute tiles with virtual tile 16 (RAPIDS) and 17 (HIVE-C)

Routing implications: - TDEST = 16 routes to the RAPIDS DMA engine - TDEST = 17 routes to the HIVE-C status aggregator - From HIVE-C: CDA packets (TUSER=2'b01) route to RAPIDS - From tiles: PKT_STATUS (TUSER=2'b11) routes to HIVE-C

4.6 Packet Type Usage by Component

Packet type usage by component

Component	Sends	Receives	Purpose
HIVE-C	CDA (2'b01), PKT_CONFIG (2'b10)	PKT_STATUS (2'b11)	Descriptor injection, network config, monitoring
RAPIDS	PKT_DATA (2'b00)	CDA (2'b01), PKT_DATA (2'b00)	DMA data transfers
Compute Tiles	PKT_DATA (2'b00)	PKT_DATA (2'b00), PKT_CONFIG (2'b10)	Computation, reconfiguration
SERV Monitors	PKT_STATUS (2'b11), PKT_CONFIG (2'b10)	PKT_CONFIG (2'b10)	Traffic reporting, tile control
Delta Routers	-	PKT_CONFIG (2'b10)	Routing table updates

Validation rules (checkable by protocol checkers at tile interfaces): - HIVE-C never sends PKT_DATA or PKT_STATUS - RAPIDS never sends CDA, PKT_CONFIG, or PKT_STATUS - Compute tiles never send CDA, PKT_CONFIG, or PKT_STATUS - SERV monitors send PKT_STATUS and PKT_CONFIG only

---

4.7 Key Architectural Features

1. Hierarchical control. HIVE-C is the single global coordinator; the 16 SERV monitors are lightweight agents. One powerful core where decisions are made, sixteen tiny cores where the data is.

2. Inband control. HIVE-C pushes descriptors as CDA packets through the Delta data fabric — no polling, no sideband descriptor bus. The control path reuses the network that already exists.

3. Distributed monitoring. Each SERV tracks its local router's traffic and autonomously injects PKT_STATUS on trigger conditions (congestion, errors, periodic reporting). HIVE-C aggregates rather than polls.

4. Reconfigurable topology. Four pre-loaded routing contexts (systolic, mesh, tree, custom) switch atomically in 10-20 cycles under the Configuration Manager's quiesce protocol.

5. Zero data-path ownership. HIVE carries control and status traffic only; the Delta data plane and the RAPIDS AXI4 path never traverse HIVE logic.

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Subsystem Architecture

HIVE consists of four primary functional blocks. Each has one clear purpose; together they provide distributed control and monitoring for the RAPIDS/Delta system.

5.1 Block 1: HIVE-C Master Controller

Single VexRiscv RISC-V processor serving as the global coordinator.

Key specifications:

HIVE-C key specifications

Attribute	Value
Core	VexRiscv “Small” configuration
ISA	RV32IM (no compressed, no floating point)
Pipeline	5-stage
Frequency	100 MHz (PoC), 150+ MHz (optimized target)
Memory	32 KB tightly-coupled I-mem + 32 KB tightly-coupled D-mem (BRAM)
External interfaces	UART/AXI4-Lite slave (host), AXIS master (CDA injection), control network master (SERVs)

Responsibilities: 1. Parse high-level transfer commands from the host 2. Generate RAPIDS DMA descriptors (256-bit CDA packets) 3. Inject descriptors into Delta via AXIS (TUSER=2’b01, TDEST=16) 4. Aggregate traffic statistics from the 16 SERV monitors 5. Manage network reconfiguration sequences 6. Handle interrupt routing and exception processing

Performance targets: descriptor injection < 30 cycles (HIVE-C → RAPIDS); command response < 1 µs @ 100 MHz; all-16-SERV status aggregation < 100 cycles.

5.2 Block 2: SERV Monitors (16 instances)

Lightweight per-tile traffic monitors built on SERV bit-serial RISC-V cores.

Key specifications (per instance):

SERV monitor key specifications

Attribute	Value
Core	SERV (Small Embedded RISC-V)
ISA	RV32I, bit-serial execution (1 bit/cycle)
Frequency	50-100 MHz (shared `serv_clk`)
Resources	~125 LUTs, 164 FFs, 1 BRAM (2 KB I-mem + 2 KB D-mem)
Interfaces	Control network slave (from HIVE-C), AXIS master (PKT_CONFIG/PKT_STATUS), router monitoring taps

Responsibilities (per tile): 1. Monitor local router traffic (packet counts, buffer occupancy) 2. Detect congestion and anomalies against programmable thresholds 3. Generate inband control packets on trigger conditions 4. Inject configuration updates into the local tile 5. Report statistics to HIVE-C via the control network 6. Support debug/trace packet injection

Monitored metrics (exposed to firmware): - Counters (32-bit, wrapping): `pkt_rx_count[4]`, `pkt_tx_count[4]` (per direction N/S/E/W), `pkt_local_inject`, `pkt_local_consume`, `buffer_overflow_count`, `stall_cycles` - Real-time: `buffer_occupancy[4]` (8-bit per direction), `congestion_flag`, `error_flags` (parity, protocol violations)

Performance targets: monitoring overhead < 5% of compute time; congestion detection < 10 cycles; periodic reporting interval 1000 cycles (configurable).

5.3 Block 3: Control Network

Dedicated low-bandwidth infrastructure for HIVE-C ↔️ SERV command and status exchange. Deliberately separate from the Delta data path: control traffic must not contend with compute traffic, and must remain live during data-plane quiesce.

Control network key specifications

Attribute	Value
Topology	Star (HIVE-C master, 16 SERV slaves)
Data width	32 bits (commands and status responses)
Arbitration	Round-robin (SERV → HIVE-C uplink)
Protocol	Valid/ready handshake, in-order, lossless

Responsibilities: 1. Deliver control commands from HIVE-C to individual SERV monitors 2. Aggregate status reports from SERVs to HIVE-C 3. Provide reliable, in-order delivery (no packet loss) 4. Prioritize urgent alerts over periodic statistics

Communication patterns: - HIVE-C → SERV (multicast or unicast): configuration commands, threshold updates, firmware control (soft reset, debug mode) - SERV → HIVE-C (uplink): periodic statistics (every 1000 cycles), congestion alerts (immediate), error notifications

Performance targets: command delivery < 10 cycles; status aggregation < 20 cycles across all 16 SERVs; round-robin fairness (no SERV starvation).

5.4 Block 4: Configuration Manager

Manages virtual configuration contexts and orchestrates network topology switching.

Configuration Manager key specifications

Attribute	Value
Virtual contexts	4 pre-loaded routing configurations
Context storage	BRAM-based context memory (routing tables per tile)
Switching protocol	Atomic context switch with packet draining (quiesce)
Distribution	PKT_CONFIG broadcast via Delta Network

The four contexts:

Virtual configuration contexts

Context	Name	Character
0	Systolic	Nearest-neighbor N/S/E/W fixed routing; router bypass; 1-2 cycles/hop
1	Packet-switched mesh	Full XY dimension-ordered routing; credit flow control; 2 virtual channels; 3-4 cycles/hop
2	Tree reduction	Hierarchical aggregation; tiles 12-15 as roots; E-W reduction tree
3	Custom/debug	User-programmable routing tables; per-tile independent configuration

Responsibilities: 1. Store multiple virtual configuration contexts 2. Coordinate atomic topology switching across all tiles 3. Manage the quiesce protocol (drain in-flight packets) 4. Broadcast configuration updates via PKT_CONFIG packets 5. Verify successful context-switch completion

Switching sequence (detailed protocol in §5.4): `CONFIG_PREPARE` → drain/quiesce (5-10 cycles) → `SET_ROUTING_MODE` broadcast → `CONFIG_ACTIVATE`. Total latency 10-20 cycles, deterministic; budgeted < 25 cycles including quiesce.

5.5 Inter-Block Communication

Inter-block communication summary

Path	Medium	Direction	Bandwidth	Latency
HIVE-C ↔️ SERVs	Control Network	Bidirectional	Low (32-bit command/status)	< 10 cycles
HIVE-C → Delta	AXIS (CDA)	Master	High (256-bit descriptors)	~10-20 cycles to RAPIDS
SERVs → Delta	AXIS (PKT_CONFIG/STATUS)	Master (per SERV)	Medium (128-bit)	~10-20 cycles through Delta
Config Manager → routers	PKT_CONFIG broadcast	Broadcast	Low (infrequent)	< 5 cycles

5.6 Resource Budget (NexysA7 100T)

FPGA target: Xilinx Artix-7 XC7A100T-1CSG324C — 63,400 LUTs, 126,800 FFs, 240 DSPs, 135 BRAM (36Kb) blocks.

Resource budget, NexysA7 100T

Component	LUTs	FFs	DSPs	BRAM (36Kb)	Notes
Block 1: HIVE-C Controller	1,400	900	0	8	VexRiscv “Small” config
Block 2: SERV Monitors (16×)	2,000	2,624	0	16	~125 LUTs per SERV
Block 3: Control Network	1,200	800	0	4	HIVE-C ↔️ SERV comm
Block 4: Configuration Manager	1,500	1,200	0	2	Virtual contexts
HIVE control plane subtotal	**6,100**	**5,524**	**0**	**30**	9.6% LUT, 22% BRAM
Delta Network 4×4 mesh (16 routers)	8,000	6,000	0	32	Delta-owned; see Delta HAS
HIVE + NoC total	**14,100**	**11,524**	**0**	**62**	22% LUT, 46% BRAM
Available for compute/RAPIDS	49,300	115,276	240	73	Remaining resources

> **Correction note (v0.1):** earlier drafts presented the 14,100-LUT subtotal as “HIVE” while omitting the Delta NoC row from the table — the NoC’s 8,000 LUTs / 6,000 FFs / 32 BRAMs were silently included in the subtotal. The table above separates the HIVE control plane (6,100 LUTs) from the

Delta NoC it configures, then gives the combined figure. The “22% LUT / 46% BRAM” occupancy that motivated the PoC budget refers to the combined figure.

Key design constraints: - HIVE consumes **zero DSP blocks** (all 240 available to compute) - HIVE uses no AXI4 master of its own — it never touches DDR directly - The combined control plane leaves **78% of logic resources** for RAPIDS and Delta compute elements

---

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 Top-Level Interfaces

`hive_top` is the subsystem boundary. Signal groups below are architectural — final port names are fixed with the RTL (see MAS, Chapter 3); integrators should treat the *classes* here as the contract.

6.1 External Interface Summary

External interface summary

Interface	Type	Width	Purpose
Host interface	Slave	32-bit	UART and/or AXI4-Lite for host commands, status, and firmware load
AXIS CDA TX	Master	256-bit	CDA descriptor injection toward RAPIDS (via Delta)
AXIS SERV TX (16×)	Master	128-bit	PKT_CONFIG / PKT_STATUS injection from each SERV monitor
AXIS HIVE-C RX	Slave	128-bit	PKT_STATUS aggregation return into HIVE-C
Control Network	Internal	32-bit	HIVE-C ↔️ SERV communication (Block 3)
Router monitor taps (16×)	Internal	per Delta	Buffer occupancy, packet counts, error flags from each tile router

6.2 Communication Protocol Rule (owner, 2026-10-13)

Everything that crosses a block boundary toward the fabric is a stream; only register files are memory-mapped.

- **AXI4-Stream** carries all packet traffic: CDA injection, SERV PKT_CONFIG/PKT_STATUS, and the status return path. AXIS is delta's native interface, and the `tuser`/`tdest`/`tid` packet classification (§4.2) with the ~10-20 cycle inband latency target only exist on the stream path.
- **AXI4-Lite** is used exclusively for register windows: the host interface and HIVE-C's AXIL master window to RAPIDS/Delta/SERV configuration.
- **No AXI4 full anywhere in HIVE.** HIVE never touches DDR and moves no burst traffic; the full protocol buys nothing here.
- The internal control network is **AXIS-shaped** too (32-bit frames on `tvalid/tready/tlast`, `tuser[0]` = urgent flag) so the repo stream BFMs and the Ocelus monitor system cover it without bespoke protocol work (MAS §2.4).

6.3 Host Interface

Two physical options, one command set:

- **UART** — serial console-style command channel; default on the PoC board (NexysA7 USB-UART bridge). Fixed 115200 8N1 in the PoC; baud is a firmware configuration value, not a synthesis parameter.
- **AXI4-Lite slave** — memory-mapped command/status window for SoC integration (e.g., a Zynq or PCIe-attached host). Same command encodings as the UART path; selection is a synthesis-time option, not runtime.

The host issues high-level transfer commands (“matmul layer N, tiles 0-7”); HIVE-C decomposes them into descriptor batches. The host never writes RAPIDS registers directly — all DMA control flows through HIVE-C firmware.

6.4 AXIS CDA TX (HIVE-C → Delta)

- **Width:** 256-bit data beats, one descriptor per packet, `tlast` on every beat (descriptors are single-beat).
- **Sideband:** `tuser[1:0]` = 2'b01 (CDA) on every beat; `tdest` = 16 (RAPIDS virtual tile) for descriptor packets.
- **Rate:** sustained injection must not stall the descriptor engine; target is back-to-back descriptor issue with < 30-cycle HIVE-C → RAPIDS latency.

6.5 AXIS SERV TX (16×, per-monitor injection)

- **Width:** 128-bit beats; packets are single-beat in the common case.
- **Sideband:** `tuser[1:0]` = 2'b10 (PKT_CONFIG) or 2'b11 (PKT_STATUS); `tdest` = target tile for config writes, 17 (HIVE-C) for status packets routed back through Delta where applicable, or local-tile distribution as defined by the Configuration Manager.
- **TID:** SERV monitors own their `tid` for identification in status traffic; the hardware TID-override rule (§4.2) does not apply to SERV-originated packets — only to compute-element PKT_DATA.

6.6 AXIS HIVE-C RX (status return)

- **Width:** 128-bit beats.
- **Purpose:** receives PKT_STATUS traffic addressed to HIVE-C (virtual tile 17) through the Delta fabric, where the source elected Delta routing rather than the control network (e.g., aggregated tile-level status).

6.7 Control Network (internal)

Star topology, HIVE-C master / 16 SERV slaves. 32-bit command/status frames with valid/ready handshake; in-order, lossless delivery. Frame layout is an implementation decision and lives in the MAS (Chapter 3); the architectural guarantees are the latency targets and the ordering/fairness contract in Chapter 3 above.

6.8 Router Monitor Taps (internal)

Each SERV monitor observes its local tile router through a passive tap — buffer occupancy per direction, packet counters, error flags. Taps are read-only observation; the SERV cannot inject values into the router data path. SERV-originated *actions* travel as PKT_CONFIG packets through the normal injection path, never through the tap.

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 Packet Formats and Routing Rules

All HIVE control traffic is AXIS packets classified by `tuser[1:0]`. These encodings are shared with the Delta Network contract — Delta owns the routing, HIVE owns the packet classes and payloads.

7.1 AXIS Packet Classification

```
// AXIS TUSER[1:0] encoding
typedef enum logic [1:0] {
    PKT_DATA   = 2'b00,  // Compute data (activations, weights,
results)
    PKT_CDA    = 2'b01,  // DMA descriptor ("CDA", Compute Direct
Access)
    PKT_CONFIG = 2'b10,  // Configuration command
    PKT_STATUS = 2'b11   // Status/monitoring data
} packet_type_t;
```

> **Naming note:** early drafts called 2'b01 `PKT_DESC`; current docs and the Delta contract call it CDA. `PKT_CDA` is the spelling to use in RTL.

Routing rules:

- `PKT_DATA` — routed through the Delta mesh to compute tiles
- `PKT_CDA` — routed to the RAPIDS DMA descriptor queue (virtual tile 16)
- `PKT_CONFIG` — routed to tile configuration registers / router config
- `PKT_STATUS` — routed to the HIVE-C monitoring aggregator (virtual tile 17)

7.2 CDA Descriptor Format (256-bit)

RAPIDS descriptors injected by HIVE-C (one per packet, single beat):

```
Bits [255:192]: Source Address (64-bit)
Bits [191:128]: Destination Address (64-bit)
Bits [127:96]:  Transfer Length (32-bit, in bytes)
Bits [95:64]:   2D Stride Configuration (32-bit)
Bits [63:32]:   Control Flags (32-bit)
Bits [31:0]:    Next Descriptor Pointer (32-bit)

Control Flags [31:0]:
  [31:28] - Burst Size (AXI beats)
  [27:24] - Burst Type (FIXED/INCR/WRAP)
  [23:20] - Priority Level (0-15)
  [19:16] - Destination Tile ID
  [15:12] - Source Tile ID (S2MM channel select; see below)
  [11:8]  - Completion Interrupt Vector
  [7]     - Enable 2D Transfer
  [6]     - Enable Scatter-Gather (follow next ptr)
  [5]     - Generate Completion Interrupt
  [4]     - Cache Coherent Access (reserved in PoC; no coherence
fabric)
  [3:0]   - Descriptor Type (future expansion)
```

: CDA descriptor field map

Channel-aware programming: for S2MM (stream-to-memory) descriptors, the source tile ID field selects which RAPIDS S2MM channel the descriptor arms; RAPIDS matches that channel against the `tid` of incoming result packets from the tiles. HIVE-C must program S2MM descriptors with the exact tile IDs it expects results from — the identity mapping `channel = tile_id` is the PoC convention.

7.3 PKT_CONFIG Format (128-bit)

```
Bits [127:96]:  Target Tile Mask (32-bit, one bit per tile)
Bits [95:64]:   Configuration Command (32-bit)
Bits [63:0]:    Payload Data (64-bit)

Command Types:
  0x0000_0001: Set Routing Mode (payload = context ID)
  0x0000_0002: Update Router Table Entry
  0x0000_0004: Set Traffic Priority Weights
  0x0000_0008: Enable/Disable Tile
  0x0000_0010: Flush Local Buffers
  0x0000_0020: Reset Statistics Counters
  0x0000_0040: Update Clock Divider
  0x0000_0080: Set Debug Trace Mode
```

: PKT_CONFIG field map

The target tile mask allows a single broadcast packet to configure multiple tiles simultaneously — the mechanism that makes atomic context switching possible (all routers update mode in the same cycle).

7.4 PKT_STATUS Format (128-bit)

```
Bits [127:120]: Source Tile ID (8-bit)
Bits [119:112]: Status Type (8-bit)
Bits [111:64]:  Timestamp (48-bit cycle counter)
Bits [63:0]:    Payload (type-dependent)

Status Types:
  0x01: Traffic Statistics (packet counts)
  0x02: Buffer Occupancy Report
  0x04: Congestion Alert
  0x08: Error/Exception Report
  0x10: Performance Counter Snapshot
  0x20: Debug Trace Data
```

: PKT_STATUS field map

The 48-bit timestamp is in the producer's clock domain; HIVE-C interprets status freshness by interval, not by absolute timestamp comparison across domains.

7.5 TID Enforcement Rule

When compute elements send PKT_DATA back toward RAPIDS, the tile Network Interface **overrides `tid` with the local tile ID**:

```
// Network Interface (NI) TX path for compute-element traffic
always_comb begin
    if (source == COMPUTE_ELEMENT && axis_tuser == PKT_DATA) begin
        // Override: RAPIDS must receive the true source tile ID
        axis_tx_tid_out = MY_TILE_ID[3:0];
    end else begin
        // SERV packets (CONFIG, STATUS) pass TID through
        axis_tx_tid_out = axis_tx_tid_in;
    end
end
```

This guarantees four properties: 1. Tiles cannot spoof source ID (security and correctness) 2. RAPIDS always receives the accurate `tid` for S2MM channel routing 3. Compute elements need not know their own tile ID 4. SERV monitors retain free use of `tid` for status identification

Protocol checkers at tile boundaries should flag any compute-element PKT_DATA whose `tid` disagrees with the egress port's tile ID.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 Memory Map and Register Spaces

Three memory-mapped views exist: the HIVE-C firmware address space, the router configuration registers (written via PKT_CONFIG or HIVE-C's AXIL master), and the soft-reset register. All offsets are byte addresses.

8.1 HIVE-C Firmware Memory Map

```
0x0000_0000 - 0x0000_7FFF: Instruction memory (32 KB)
0x0000_8000 - 0x0000_FFFF: Data memory (32 KB)
0x4000_0000 - 0x4000_0FFF: RAPIDS register interface (AXIL master
window)
0x4000_1000 - 0x4000_1FFF: Delta network control
0x4000_2000 - 0x4000_2FFF: SERV monitor status (16 × 256 bytes)
0x4000_3000 - 0x4000_3FFF: Configuration registers
0x4000_4000 - 0x4000_4FFF: Host interface (UART/AXI4-Lite)
```

: HIVE-C firmware memory map

The SERV status window (0x4000_2000) exposes one 256-byte record per monitor: counters, occupancy snapshot, error flags, and firmware heartbeat. Layout is fixed with the SERV firmware (see MAS Chapter 3).

8.2 Router Configuration Registers (per tile)

Each Delta tile router exposes this register block. HIVE-C reaches them through its AXI4-Lite master window (Delta network control region) or via PKT_CONFIG broadcast.

```
Base + 0x00: MODE_SELECT     (RW) - Context ID [1:0]
Base + 0x04: ROUTE_TABLE_0   (RW) - Custom routing entry 0
Base + 0x08: ROUTE_TABLE_1   (RW) - Custom routing entry 1
     ...
Base + 0x20: FLOW_CTRL_CFG   (RW) - Credit limits, VC allocation
Base + 0x24: PRIORITY_MASK   (RW) - QoS priority weights
Base + 0x28: STATUS          (RO) - Buffer occupancy, active packets
Base + 0x2C: STATISTICS      (RO) - Packet counters
Base + 0x30: ERROR_FLAGS     (RO) - Overflow, underflow, parity errors
Base + 0x34: DEBUG_TRACE     (RW) - Trace filter configuration
```

: Router configuration register map

ROUTE_TABLE_n blocks are only meaningful in Context 3 (custom/debug); contexts 0-2 use hardwired routing functions, and the table registers read back as zero.

8.3 Soft Reset Register

HIVE supports selective soft resets from configuration space (offset 0x0000 in the HIVE configuration region):

```
Bits [31:16] - SERV Reset Mask (one bit per SERV monitor)
Bit  [8]     - Delta Network Soft Reset
Bit  [4]     - Control Network Soft Reset
Bit  [0]     - HIVE-C Soft Reset (use with caution)
```

: Soft reset register

Write 1 to assert, 0 to deassert. Soft resets are synchronous to the target clock domain and follow the reset sequencing rules in the MAS clocking chapter. Use cases: SERV hang recovery without system disruption, clearing Delta state during reconfiguration, and component-level debug.

8.4 Fabric IDLE Status Register

A read-only status register in the HIVE configuration region carries the aggregated **fabric IDLE** signal — the quiescent indication from the Delta fabric covering all participating routers (and the SERV injection paths). HIVE-C firmware reads it to verify quiesce before issuing `SET_ROUTING_MODE` (§5.4), and the host may read it for bring-up diagnostics. The register is part of the PeakRDL-managed HIVE register bank (generated from `rdl/` sources per the repo convention, see MAS §2.5): bit [0] = fabric idle, bits [16:1] reserved (per-router idle granularity is a possible future extension).

8.5 Register Access Rules

- **Host → HIVE-C:** host interface only; the host has no path to RAPIDS, Delta, or SERV registers except through HIVE-C firmware actions.
- **HIVE-C → RAPIDS/Delta/SERV:** AXI4-Lite master window and PKT_CONFIG broadcast; descriptor *data* never travels over the register path — it travels as CDA packets.
- **SERV → registers:** SERV monitors write tile configuration only via PKT_CONFIG injection; their router tap is observation-only.

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 HIVE-C Firmware Architecture

HIVE-C runs bare-metal RV32IM firmware. The architecture below is the reference structure; module boundaries are a firmware convention, not an RTL constraint.

```
main.c              - Initialization and main loop
   ├── rapids_mgr.c  - DMA descriptor management
   ├── config_mgr.c  - Network reconfiguration
   ├── stats_mgr.c   - Performance monitoring aggregation
   ├── interrupt.c   - IRQ handlers (RAPIDS completion, SERV alerts)
   └── host_if.c     - UART/AXI4-Lite command interface
```

: HIVE-C firmware module structure

Responsibilities by module:

Firmware module responsibilities

Module	Responsibility
`main.c`	Hardware init, context load, main loop
`rapids_mgr.c`	Descriptor pool, batching, channel-aware S2MM programming
`config_mgr.c`	Context selection, quiesce/activate orchestration
`stats_mgr.c`	SERV status collection, congestion policy, performance model updates
`interrupt.c`	RAPIDS completion, SERV alert routing, error handling
`host_if.c`	Command parsing, response formatting, firmware load support

9.1 Main Loop

```
void hive_main_loop(void) {
    init_hardware();
    load_network_config(CONTEXT_MESH); // Start in mesh mode (Context
1)

    while (1) {
        if (host_command_pending()) {
            handle_host_command();
        }
        if (timer_expired(STATS_INTERVAL)) {
            collect_serv_statistics();
            update_performance_model();
        }
        if (congestion_detected()) {
            adaptive_route_balancing();
        }
        if (rapids_ready() && desc_queue_has_work()) {
            issue_descriptor_batch(MAX_INFLIGHT);
        }
        if (irq_pending()) {
            process_interrupts();
        }
    }
}
```

The loop is poll-plus-interrupt: fast paths (descriptor issue) poll, slow paths (statistics, host commands) share the same iteration. No RTOS; latency comes from staying in this loop.

9.2 Descriptor Injection API

```
// Inject one 256-bit CDA descriptor toward RAPIDS (virtual tile 16).
// Blocks until the AXIS master accepts the beat.
void inject_descriptor(const descriptor_t *desc);
```

`inject_descriptor` is the single path by which descriptors reach RAPIDS — firmware never writes RAPIDS descriptor registers directly. The injection firmware sets `tuser = PKT_CDA`, `tdest = 16`, and waits for `tready`.

9.3 Interrupts

HIVE-C interrupt inventory

Source	Signal	Handler action
RAPIDS	Descriptor completion	Bookkeeping, next-batch issue
RAPIDS	Error	Log, halt or degrade per policy
SERV (via control network)	Congestion alert	Policy decision: throttle, reroute, or context switch
SERV (via control network)	Error notification	Mark tile, notify host
Host interface	Command ready	Wake command parser

Last Updated: 2026-10-12

10 Descriptor Scheduling

HIVE-C's scheduler admits descriptor **chains** — descriptors linked through the next-descriptor pointer with scatter-gather enabled (HAS §4.2) — and schedules the **kick**: when each chain starts, and on which RAPIDS channel. The kick/flow split is deliberate (owner, 2026-10-13): HIVE-C owns admission and kick ordering; **RAPIDS' built-in flow control owns pacing** — producers and consumers (e.g., one cell producing, another consuming) synchronize through hardware handshakes via flags in memory, so a kicked chain executes at the rate the producer/consumer pair sustains without HIVE-C polling.

Scheduling policy orders the kicks:

10.1 Priority Levels

Descriptor priority classes

Level	Class	Examples
0 (highest)	Real-time / latency-sensitive	Small control transfers, urgent spills
1	Standard compute data movement	Activation/weight loads, result drains
2	Background / prefetch	Next-layer prefetch, cache warming
3 (lowest)	Debug / trace	Trace dumps, instrumentation

10.2 Scheduling Policy

1. Round-robin within a priority level
2. Strict priority across levels: a non-empty higher level preempts issue (already-injected descriptors are never reordered)
3. Age-based promotion: a descriptor waiting 1000 cycles promotes one level
4. Deadline-aware tie-break for level-0 descriptors

10.3 Channel-Aware S2MM Programming

The critical correctness rule: **S2MM descriptors must name the tile that will send the results**, because RAPIDS matches incoming PKT_DATA `tid` against armed S2MM channels.

```
// Direct mapping: Tile N sends with TID=N, routes to Channel N
#define TILE_TO_CHANNEL(tile_id)  (tile_id)  // Identity mapping

void program_s2mm_descriptor(uint8_t source_tile) {
    descriptor_t desc;
    desc.type         = DESC_S2MM;                  // Network →
Memory
    desc.src_addr     = 0;                          // Not used for
S2MM
    desc.dst_addr     = result_buffer[source_tile]; // DDR write
address
    desc.length       = result_size;
    desc.source_tile  = source_tile;                //
Descriptor[15:12]
    desc.dest_tile    = 16;                         // RAPIDS virtual
tile
    inject_descriptor(&desc);
    // RAPIDS arms S2MM Channel[source_tile]; a PKT_DATA with tid ==
source_tile
    // consumes this descriptor.
}
```

10.4 Worked Example: Matrix-Multiply Layer

```
void execute_matmul_layer(uint8_t num_tiles) {
    // Phase 1: send activations to all participating tiles (MM2S)
    for (int tile = 0; tile < num_tiles; tile++) {
        descriptor_t mm2s_desc;
        mm2s_desc.type        = DESC_MM2S;          // Memory →
Network
        mm2s_desc.src_addr    = activation_buffer[tile];
        mm2s_desc.dst_addr    = 0;                  // Not used for
MM2S
        mm2s_desc.length      = activation_size;
        mm2s_desc.dest_tile   = tile;               // TDEST routing
to tile
        mm2s_desc.source_tile = 0;                  // Not used for
MM2S
        mm2s_desc.priority    = 0;
        inject_descriptor(&mm2s_desc);
    }

    // Phase 2: pre-program S2MM descriptors for result collection.
    // CRITICAL: one per expected source tile, channel = tile_id.
    for (int tile = 0; tile < num_tiles; tile++) {
        program_s2mm_descriptor(tile);
    }

    wait_for_all_tiles_complete();
}
```

If a result arrives with a `tid` that matches no armed channel, RAPIDS flags it; HIVE-C's error policy (log, halt, or degrade) applies. A missing S2MM descriptor is a firmware bug, not a hardware fault — this is the channel-mapping invariant the firmware must maintain.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 SERV Monitor Programming

Each SERV runs a small monitor firmware image (hand-optimized RV32I assembly is expected; a SERV executes one bit per cycle, so instruction count directly buys reaction latency). All sixteen instances run the same image; per-tile identity comes from a read-only tile-ID register the wrapper exposes at address 0xF000_0000.

11.1 Trigger Conditions

The monitor firmware evaluates five trigger classes:

SERV trigger conditions

#	Trigger	Default threshold	Action
1	Congestion	`buffer_occupancy > 75%`	Send PKT_STATUS congestion alert to HIVE-C; optionally throttle local injection
2	Error	parity error or protocol violation	Send error status packet; halt local injection pending recovery
3	Periodic report	1000 cycles	Send statistics snapshot to HIVE-C
4	Descriptor queue event	RAPIDS completion signal	Notify HIVE-C for next-batch scheduling
5	Custom	user-defined via debug registers	Inject debug trace packet

Thresholds, intervals, and enable bits are configurable by HIVE-C over the control network at runtime.

11.2 Inband Packet Injection

```
// SERV code to inject a packet through the AXIS TX FIFO.
// Example: 2D transfer descriptor (PKT_CDA) — descriptors may be
// injected by SERVs as well as by HIVE-C.
void serv_inject_packet(uint32_t *words, uint8_t nwords, uint8_t
pkt_type,
                        uint8_t tdest) {
    for (int i = 0; i < nwords; i++) {
        AXIS_TX_FIFO[i] = words[i];
    }
    AXIS_TX_TUSER  = pkt_type;   // 2'b10 PKT_CONFIG, 2'b11
PKT_STATUS, ...
    AXIS_TX_TDEST  = tdest;
    AXIS_TX_TLAST  = 1;
    AXIS_TX_TVALID = 1;
    while (!AXIS_TX_READY) { }   // Wait for handshake
    AXIS_TX_TVALID = 0;
}
```

The TX FIFO abstracts the AXIS master; depth and handshaking are implemented in the wrapper (MAS Chapter 2). A SERV should never block on `tready` while holding time-critical counters — the reference firmware snapshots metrics first, then transmits.

11.3 Reporting Discipline

- **Periodic snapshots** ride the control network uplink (32-bit frames, round-robin arbitrated).
- **Urgent alerts** (congestion, errors) may use either the control network fast path or a PKT_STATUS injection through Delta, per HIVE-C's runtime configuration; the control network is preferred during data-plane quiesce.
- **Timestamps** are local `serv_clk` cycle counts; HIVE-C treats them as sequence indicators, not comparable clocks.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

12 Network Reconfiguration Workflow

Network topology switching is HIVE-C firmware driving the Configuration Manager hardware through a fixed three-command protocol. The sequence is identical every time; only the context ID changes.

12.1 Atomic Switch Protocol

```
1. HIVE-C issues CONFIG_PREPARE
   - Target tiles flush in-flight packets
   - Routers enter quiescent state (5-10 cycles)

2. HIVE-C broadcasts SET_ROUTING_MODE (PKT_CONFIG, command
0x0000_0001)
   - Target tile mask = all participating tiles
   - Payload = context ID (0-3)
   - All tiles simultaneously update MODE_SELECT

3. HIVE-C issues CONFIG_ACTIVATE
   - Tiles resume operation under the new routing
```

Total latency 10-20 cycles, deterministic; budget < 25 cycles including quiesce. Overhead target < 1% for workloads whose phases run 1000+ cycles.

12.2 Worked Flow: Mesh to Tree Reduction

1. Host command (or scheduler policy): “switch to tree reduction mode”
2. HIVE-C broadcasts `CONFIG_PREPARE` over the control network; SERV monitors quiesce their local injection
3. Routers drain in-flight packets (5-10 cycles; SERV taps confirm occupancy at zero via the STATUS registers)
4. HIVE-C broadcasts `SET_ROUTING_MODE`, payload = context 2, tile mask = all 16 tiles — one PKT_CONFIG packet updates every router in the same cycle
5. HIVE-C issues `CONFIG_ACTIVATE`; the network resumes with the E-W reduction tree active

12.3 Safety Rules

- **Never switch without quiesce.** A `SET_ROUTING_MODE` issued while packets are in flight is a protocol violation; the Configuration Manager does not hardware-enforce the drain, so firmware must verify the fabric IDLE status register (below) before step 2. A hardware interlock is optional future hardening (MAS §2.5).
- **Context 3 tables must be loaded before activation.** Custom routing entries are written via `Update Router Table Entry` (0x0000_0002) packets while the router is quiescent.
- **Rollback:** on `CONFIG_ACTIVATE` failure (tile error flags), firmware re-runs the sequence back to the previous context; there is no partial commit by construction (single-cycle mode register update).

Last Updated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 Performance Characteristics

All values in this chapter are **targets**, not measured results — they are the acceptance criteria RTL bring-up will be verified against.

13.1 Latency Targets

Latency targets

Metric	Target	Notes
Descriptor injection (HIVE-C → RAPIDS)	< 30 cycles	Includes AXIS handshake + Delta traversal
CDA packet latency through Delta	~10-20 cycles	One hop per router; tile 16 is edge-attached
Command response (host → HIVE-C action)	< 1 µs @ 100 MHz	Host interface round trip
Context switch (incl. quiesce)	< 25 cycles	Deterministic; 10-20 typical
Congestion detection (occupancy → alert sent)	< 10 cycles	SERV trigger evaluation + packetize
Status packet generation	< 5 cycles	From trigger to `tvalid`
SERV aggregation (all 16 via control network)	< 100 cycles	HIVE-C-side collection; per-uplink < 20

13.2 Throughput and Overhead Targets

Throughput and overhead targets

Metric	Target
Delta Network throughput	≥ 80% of theoretical maximum
SERV monitoring overhead	< 5% of compute time
Context-switch overhead	< 1% for 1000+ cycle phases
Descriptor processing	Continuous, no stalls at full injection rate

13.3 Design Tradeoffs

Virtual channels per router (Delta-owned; HIVE is the consumer that cares about deadlock freedom and latency):

VC tradeoff (per router estimates)

VCs	Area (LUTs)	Max Freq (MHz)	Deadlock Free	Latency (cycles)	Verdict
1	400	250	No	3.2	Minimal area, unsafe for XY
2	500	220	Yes (XY)	3.5	**PoC choice**
4	650	180	Yes	4.1	QoS headroom, revisit at 8×8

SERV reporting interval (how often each monitor reports):

SERV interval tradeoff

Interval (cycles)	Overhead (%)	Detection Latency	Memory (bytes)
100	8.5	Excellent	512
500	2.1	Good	256
1000	1.2	Acceptable	**PoC default (128)**
5000	0.3	Poor	64

13.4 Success Criteria

Functional: inband descriptor control demonstrated end-to-end; 16 monitors tracking accurately; context switch < 25 cycles; correct routing for all four packet classes; zero packet loss under normal operation; congestion alert < 10 cycles.

Performance: ≥ 100 MHz on NexysA7 100T; descriptor injection < 30 cycles; NoC ≥ 80% theoretical throughput; SERV overhead < 5%; switch overhead < 1% on 1000+ cycle phases.

Educational: SimPy models predict hardware within ±10%; 100% of RTL modules with passing unit tests; API docs with examples; ≥ 5 tutorials; modular replacement of any block without system redesign.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

14 Integration and Verification

14.1 RAPIDS Integration

HIVE-C connects to RAPIDS through two complementary interfaces:

Control interface (HIVE-C AXI4-Lite master → RAPIDS registers): status reads, queue base addresses, DMA parameter configuration. Used for setup and status; not on the data path.

Descriptor injection (HIVE-C AXIS master → Delta → RAPIDS): 256-bit CDA packets classified by `tuser=2'b01` and routed to the RAPIDS descriptor FIFO at virtual tile 16. No memory round-trip — 10-20 cycles vs. 100+ for memory-mapped descriptor writes. **Execution pacing:** RAPIDS' producer/ consumer flow control (memory-flag handshakes between producer and consumer endpoints) paces chains after the kick — one cell can produce while another consumes without HIVE-C polling per-chain progress; HIVE-C tracks completion via the interrupt inventory (§5.1).

The contract with RAPIDS is exactly the CDA descriptor format in §4.2 plus the channel-aware S2MM rule in §5.2. See the RAPIDS Beats HAS for the consumer side.

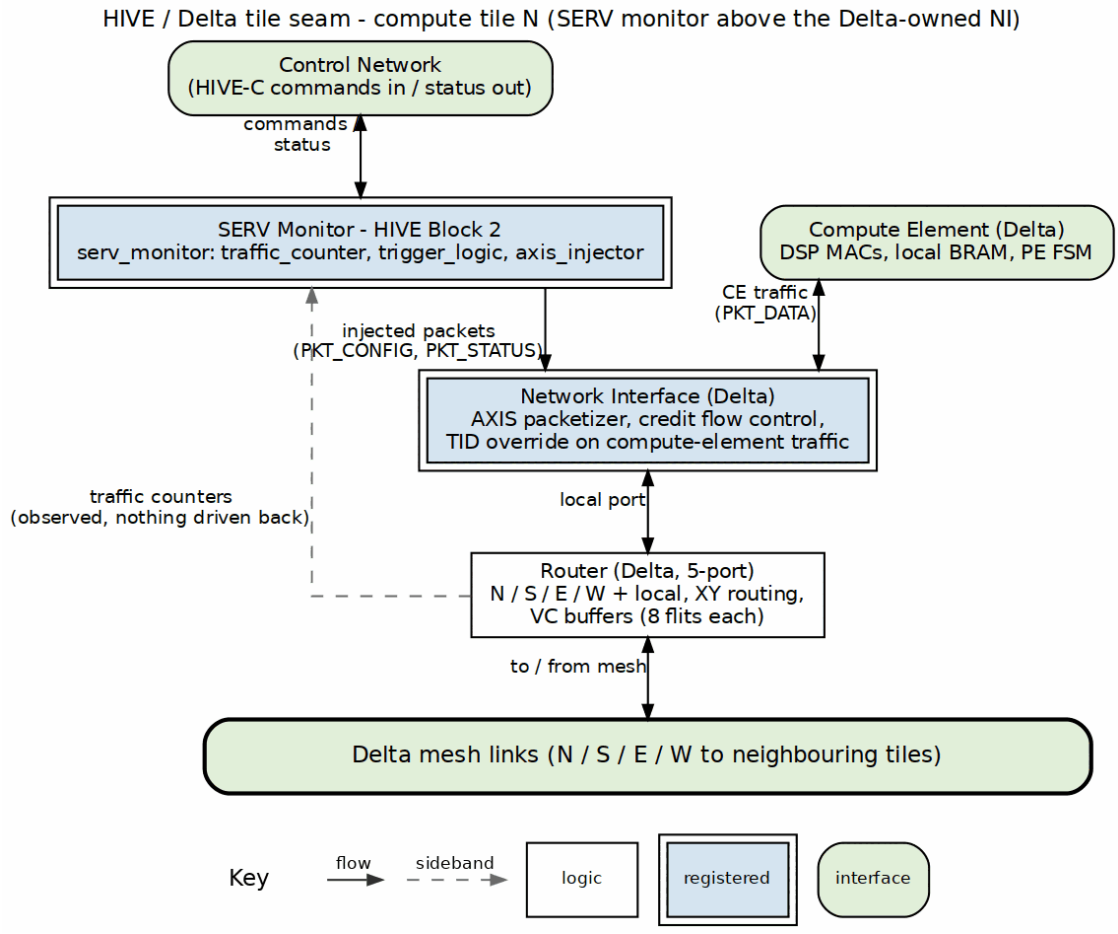
14.2 Delta Integration

HIVE and Delta meet at three seams:

1. **Packet classes** — HIVE originates CDA/PKT_CONFIG/PKT_STATUS; Delta routes them by `tuser`/`tdest` under contexts HIVE selects.
2. **Virtual tiles** — RAPIDS = 16, HIVE-C = 17, edge-attached to the mesh.
3. **Router configuration** — the register block in §4.3, written via PKT_CONFIG broadcast (Configuration Manager) or HIVE-C's AXIL window.

The tile internals (NI, router, compute element) are Delta's architecture:

14.2.1 Figure 7.1: HIVE/Delta tile seam — SERV monitor above the Delta-owned NI



HIVE/Delta tile seam: SERV monitor, network interface, router, compute element

14.3 System Top Integration

```
// Top-level integration example (architectural port names; see MAS
ch.3)
module system_top (
    input  logic        clk_100mhz,
    input  logic        rst_n,
    // Host interface
    input  logic [31:0] host_cmd,
    output logic [31:0] host_resp,
    // External memory (for RAPIDS)
    // ... AXI4 memory interface ...
);

    // HIVE instance
    hive_top u_hive (
        .aclk           (clk_100mhz),
        .aresetn        (rst_n),
        .host_cmd       (host_cmd),
        .host_resp      (host_resp),
        .axis_cda_*     (hive_to_delta_cda),
        .axis_status_*  (delta_to_hive_status)
    );

    // Delta Network instance
    delta_network_4x4 u_delta (
        .aclk           (clk_100mhz),
        .aresetn        (rst_n),
        .tile16_*       (rapids_interface),  // RAPIDS virtual tile
        .tile17_*       (hive_interface)     // HIVE-C virtual tile
    );

    // RAPIDS instance
    rapids_top u_rapids (
        .aclk           (clk_100mhz),
        .aresetn        (rst_n),
        .axis_cda_*     (rapids_interface),  // From Delta
        .m_axi_*        (memory_interface)   // To external memory
    );

endmodule
```

14.4 Verification Strategy

Level 1 — Unit (cocotb): every RTL module gets a dedicated testbench. Coverage target: 100% line, 95%+ branch. Example (SERV congestion trigger):

```
@cocotb.test()
async def test_congestion_detection(dut):
    """SERV triggers alert when buffer exceeds threshold"""
    clock = Clock(dut.clk, 10, units="ns")
    cocotb.start_soon(clock.start())
    dut.rst_n.value = 0
    await RisingEdge(dut.clk)
    dut.rst_n.value = 1

    dut.congestion_threshold.value = 120  # 75% of 160-entry FIFO
    for i in range(125):                   # Inject with backpressure
        dut.pkt_rx_valid.value = 1
        dut.pkt_rx_ready.value = 0
        await RisingEdge(dut.clk)

    await RisingEdge(dut.clk)
    assert dut.congestion_alert.value == 1
    assert dut.axis_tx_valid.value == 1
    assert dut.axis_tx_tuser.value == 0b11  # PKT_STATUS
```

Level 2 — Integration: multi-module scenarios (SERV + router, HIVE-C + control network + 16 SERVs, Configuration Manager + Delta); protocol compliance checkers; randomized traffic stress.

Level 3 — System: full HIVE + RAPIDS + Delta against real workload traces (matmul, convolution); SimPy prediction vs. RTL-measured performance (±10% acceptance).

Level 4 — Hardware: NexysA7 bring-up, ILA capture, performance counters vs. model validation.

Formal property inventory (critical properties, formal-verified):

Formal property inventory

Property	Statement
Router deadlock freedom	A request denied a grant eventually receives it (XY + 2 VC)
Descriptor FIFO ordering	Enqueue order preserved at dequeue per queue
Context switch atomicity	No packet crosses a router between `SET_ROUTING_MODE` and `CONFIG_ACTIVATE`
TID integrity	No compute-element PKT_DATA

Property	Statement
	egresses with `tid ≠ tile_id`
Quiesce completeness	`STATUS.active_packets == 0` before mode update

14.5 Implementation Roadmap

Implementation roadmap

Phase	Scope	Weeks
1 — Core infrastructure	VexRiscv integration + memories; one SERV monitor with counters; basic AXIS injection; unit tests	1-3
2 — 2×2 prototype	2×2 mesh + 4 SERVs; control network; config register bank; descriptor-injection integration test	4-6
3 — Full 4×4	16 routers + SERVs; ≥ 2 virtual contexts; switch logic; HIVE-C scheduler firmware; RAPIDS + Delta end-to-end	7-10
4 — Optimization	100 MHz closure; Artix-7 fit; SimPy-guided tuning	11-12
5 — Validation	NexysA7 hardware-in-loop; model-vs-hardware; stress; tutorials	13-14

14.6 Expansion Path (PoC → Production)

Kintex-7 325T scaling: 8×8 mesh (64 tiles), 64 SERV monitors, 4 VexRiscv cores under hierarchical control, 150-200 MHz, 8 routing contexts. Estimated control overhead: 50,000 LUTs + 288 BRAM — still leaving 153,800 LUTs and 840 DSPs for compute. Feature extensions beyond scaling: adaptive routing, quality-of-service guarantees, fault tolerance with dynamic rerouting, and ML-assisted traffic prediction on the SERV fabric.

Last Updated: 2026-10-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

15 HIVE Micro-Architecture Specification Index

Hierarchical Intelligent Vector Environment

Version: 0.3 **Date:** 2026-10-20 **Purpose:** Block-level micro-architecture specification for the HIVE distributed control and monitoring subsystem — implementation intent for the RTL that has not been written yet.

15.1 Document Organization

Note: All chapters linked below for automated document generation.

15.1.1 Front Matter

- Document Information

15.1.2 Chapter 1: Overview

- Design Philosophy and Machine Organization
- Clocking and Reset

15.1.3 Chapter 2: Functional Blocks

- hive_top
- hive_c_controller
- serv_monitor
- control_network
- config_manager
- Common Utilities

15.1.4 Chapter 3: Internal Interfaces

- Packages and Wiring

15.1.5 Chapter 4: Micro-Architecture Behavior

- The Descriptor Injection Cycle
- The Monitoring Cycle
- The Context Switch Sequence
- Reset and Boot Behavior

15.1.6 Chapter 5: Verification Hooks

- Verification Hooks and Observability

15.1.7 Chapter 6: Configuration

- Parameters, Defines, and Filelists

15.2 Related Documentation

- **HIVE HAS** - Hardware Architecture Specification (external contract)
- **HIVE PRD** - Product requirements and roadmap
- **RAPIDS Beats MAS** - the DMA engine's internals HIVE-C drives
- **Delta MAS** - router/NI internals HIVE configures and monitors
- **RISC-V Falcon Suite README** - suite overview (hive-serv moved to the compute-eng-ip family 2026-10-13)

Last Updated: 2026-10-13 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 Product Requirements Document (PRD)

16.1 HIVE - Hierarchical Intelligent Vector Environment

Version: 0.2 **Date:** 2026-10-12 **Status:** HAS/MAS v0.2 published; Phase 1 RTL in progress (3 of 5 blocks verified, see vault hive-serv TASK-000) **Owner:** RTL Design Sherpa Project **Parent Document:** `/PRD.md`

16.2 1. Executive Summary

The Hierarchical Intelligent Vector Environment (HIVE) is a distributed control and monitoring subsystem that coordinates the RAPIDS DMA engine and Delta Network. It demonstrates hierarchical RISC-V processor architecture, distributed monitoring, and dynamic network reconfiguration.

16.2.1 1.1 Quick Stats

- **Architecture:** 4 major blocks (HIVE-C, SERV monitors, Control network, Config manager)
- **Processors:** 1 VexRiscv (master) + 16 SERV (monitors) = 17 RISC-V cores
- **Interfaces:** Control Network (internal), AXIS (to Delta Network)
- **Resource Budget:** 6,100 LUTs HIVE control plane (14,100 including the Delta NoC), 30 BRAM on NexysA7 100T
- **Status:** Specification phase; HAS/MAS v0.2 published 2026-10-12

16.2.2 1.2 Subsystem Goals

- **Primary:** Demonstrate hierarchical control system design patterns
- **Secondary:** Enable dynamic network reconfiguration and distributed monitoring
- **Tertiary:** Educational reference for RISC-V integration and NoC control

---

16.3 2. Documentation Structure

This PRD provides a high-level overview. **Detailed specifications are maintained separately:**

16.3.1 HIVE Hardware Architecture Specification (HAS) — external contract

Location: `projects/components/compute-eng-ip/hive-serv/docs/hive_has/`

- **Index** - HAS structure
- System overview, subsystem architecture (4 blocks), interfaces & packet formats, programming model, performance targets, integration & verification

16.3.2 HIVE Micro-Architecture Specification (MAS) — implementation intent

Location: `projects/components/compute-eng-ip/hive-serv/docs/hive_mas/`

- **Index** - MAS structure
- Block-level module trees, clocking/reset, internal interfaces, cycle-level behavior of the key operations, verification hooks, parameters/filelists

16.3.3 Other Documentation

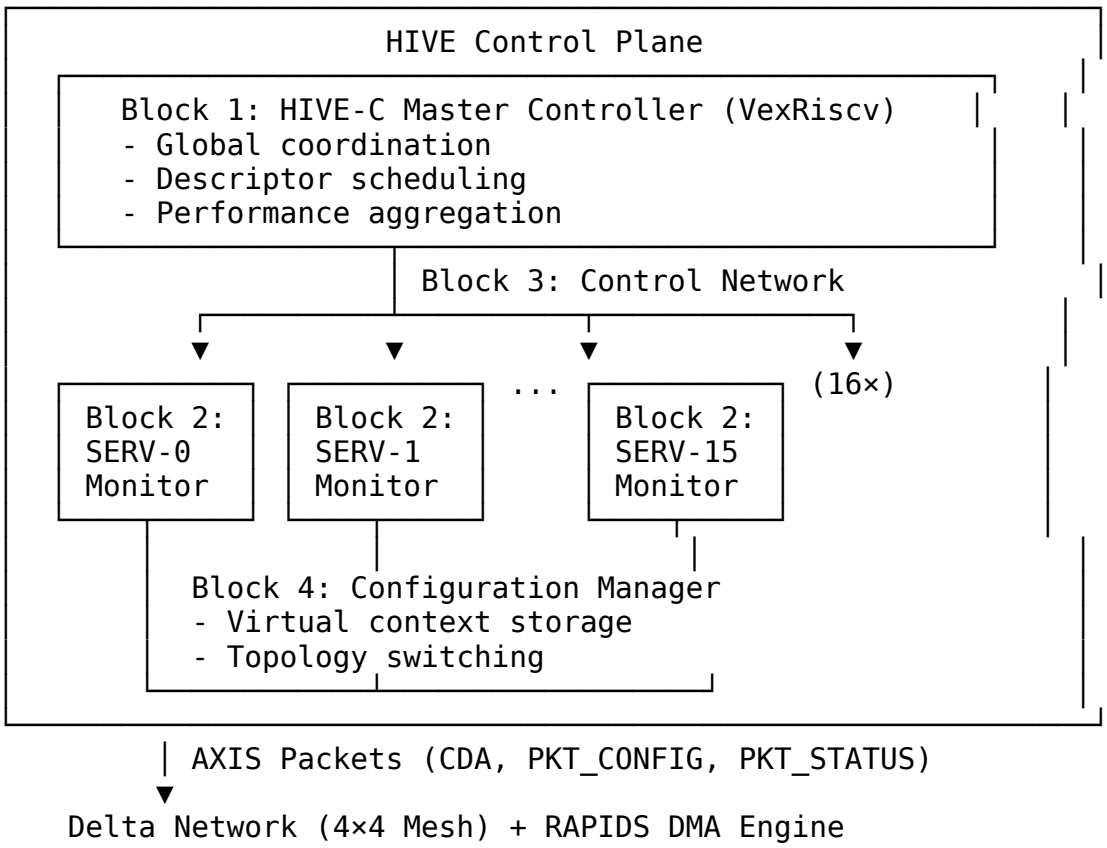
- **README** - Quick start and integration guide (to be created)
- **CLAUDE** - AI assistance guide for this subsystem
- **Tasks** - none tracked. The vault area was deleted 2026-09-27: hive has no .sv, no tests and no filelists, so its 24 open items described work that had not begun. They are in git history at `vault/tracking/projects/components/compute-eng-ip/hive-serv/`.

  History note: the earlier `docs/hive_spec/` + `docs/hive_specification.md` drafts (v0.3) were migrated into the HAS/MAS and retired 2026-10-12; they remain in git history.

16.4 3. Architecture Overview

16.4.1 3.1 Top-Level Block Diagram

HIVE System Architecture



See: `docs/hive_has/ch02_system_overview/01_system_overview.md` and `docs/hive_has/ch03_architecture/01_subsystem_architecture.md` for the architecture; `docs/hive_mas/` for block internals.

16.4.2 3.2 Resource Budget (NexysA7 100T)

FPGA Target: Xilinx Artix-7 XC7A100T-1CSG324C

Component	LUTs	FFs	DSPs	BRAM (36Kb)	Notes
Block 1: HIVE-C Controller	1,400	900	0	8	VexRiscv “Small” config
Block 2: SERV Monitors (16×)	2,000	2,624	0	16	125 LUTs per SERV
Block 3: Control	1,200	800	0	4	HIVE-C

Component	LUTs	FFs	DSPs	BRAM (36Kb)	Notes
Network					↔️ SERV comm
Block 4: Configuration Manager	1,500	1,200	0	2	Virtual contexts
HIVE control plane subtotal	**6,100**	**5,524**	**0**	**30**	9.6% LUT, 22% BRAM
Delta Network 4×4 mesh (16 routers)	8,000	6,000	0	32	Delta-owned; see Delta HAS
HIVE + NoC total	**14,100**	**11,524**	**0**	**62**	22% LUT, 46% BRAM
Available for Compute/RAPIDS	49,300	115,276	240	73	Remaining resources
Total Budget	**63,400**	**126,800**	**240**	**135**	100%

> **Correction note (2026-10-12):** earlier tables presented 14,100 LUTs as the “HIVE subtotal” while omitting the Delta NoC row; the NoC’s 8,000 LUTs were silently included. The table now separates the HIVE control plane from the Delta NoC. The 22%-LUT / 46%-BRAM occupancy figures refer to the combined total.

Key Design Constraints: - HIVE consumes **zero DSP blocks** (all 240 DSPs available for compute) - HIVE’s own memory footprint is 30 BRAM blocks (22%); combined with the NoC it is 62 (46%) - HIVE uses **9.6% of LUTs** on its own (22% combined with the NoC) - Leaves **78% of logic resources** for RAPIDS and Delta Network compute elements

---

16.5 4. Key Features

16.5.1 4.1 Hierarchical RISC-V Architecture

Feature	Status	Description
VexRiscv master (HIVE-C)	note	RV32IM, 5-stage pipeline, 32KB I-mem + 32KB D-mem
SERV monitors (16×)	note	RV32I bit-serial, 2KB I-mem + 2KB D-mem each
Control network	note	Star topology, round-robin arbitration
Independent firmware	note	Separate binaries for HIVE-C and SERV

16.5.2 4.2 Distributed Monitoring

Feature	Status	Description
Per-tile traffic counters	note	Packet counts per direction (N/S/E/W)
Buffer occupancy tracking	note	Real-time FIFO fill levels
Congestion detection	note	Programmable thresholds, immediate alerts
Error detection	note	Parity errors, protocol violations
Periodic reporting	note	Configurable interval (default 1000 cycles)

16.5.3 4.3 Network Reconfiguration

Feature	Status	Description
Virtual configuration contexts	note	4 pre-loaded routing modes
Atomic context switching	note	Deterministic latency (<25 cycles)
Quiesce protocol	note	Drain in-flight packets before switch
Broadcast	note	PKT_CONFIG packets

Feature	Status	Description
configuration		to all tiles

16.5.4 4.4 RAPIDS Integration

Feature	Status	Description
Inband descriptor injection	note	CDA packets via Delta Network to RAPIDS
Low-latency delivery	note	~10-20 cycles vs. 100+ for memory-mapped
AXIS master interface	note	Direct injection into Delta Network
Completion tracking	note	PKT_STATUS packets from RAPIDS

16.6 5. Interfaces

16.6.1 5.1 External Interfaces

Interface	Type	Width	Purpose
Host Interface	Slave	32-bit	UART/AXI4-Lite for host commands
AXIS (HIVE-C TX)	Master	256-bit	CDA packet injection to RAPIDS
AXIS (SERV TX, 16×)	Master	128-bit	PKT_CONFIG, PKT_STATUS injection
AXIS (HIVE-C RX)	Slave	128-bit	PKT_STATUS from network
Control Network	Internal	32-bit	HIVE-C ↔️ SERV communication

See: `docs/hive_has/ch04_interfaces/` for the complete interface, packet-format, and register-map specifications.

16.6.2 5.2 Virtual Tile Mapping

Virtual Tile	Entity	Connection
Tile 16	RAPIDS DMA	Edge-attached to the 4×4 mesh
Tile 17	HIVE-C Controller	Edge-attached to the 4×4 mesh

16.7 6. Use Cases

16.7.1 6.1 Descriptor Scheduling

Scenario: HIVE-C receives high-level transfer command from host

Flow: 1. Host sends command via UART/AXI4-Lite to HIVE-C 2. HIVE-C generates 256-bit CDA descriptor 3. HIVE-C injects CDA packet via AXIS master (TUSER=2'b01, TDEST=16) 4. Delta Network routes to RAPIDS (Virtual Tile 16) 5. RAPIDS descriptor engine processes 6. Completion reported via PKT_STATUS to HIVE-C

16.7.2 6.2 Congestion Management

Scenario: SERV monitor detects buffer congestion on tile 7

Flow: 1. SERV-7 monitors router buffer occupancy 2. Threshold exceeded (buffer_occupancy > cfg_threshold) 3. SERV-7 generates PKT_STATUS alert packet 4. Alert routed to HIVE-C (Virtual Tile 17) 5. HIVE-C aggregates alerts, initiates mitigation 6. HIVE-C may trigger network reconfiguration

16.7.3 6.3 Network Reconfiguration

Scenario: Switch from packet-switched mesh to tree reduction mode

Flow: 1. HIVE-C issues CONFIG_PREPARE command via control network 2. All SERV monitors flush in-flight packets 3. Routers enter quiescent state 4. HIVE-C broadcasts SET_ROUTING_MODE (PKT_CONFIG, context=2) 5. All tiles simultaneously update routing mode 6. HIVE-C issues CONFIG_ACTIVATE command 7. Network resumes with tree reduction routing

---

16.8 7. Test Coverage

16.8.17.1 Current Status

Overall: Phase 1 RTL in progress (2026-10-13)

Component	Spec Status	Implementation Status
HIVE-C Controller	HAS/MAS v0.2	Phase 1 in progress (descriptor_gen, axis_cda_master verified)
SERV Monitors	HAS/MAS v0.2	Submodules written; top pending (TASK-000)
Control Network	HAS/MAS v0.2	CN-01..03 verified; rotation scenarios blocked (TASK-000)
Configuration Manager	HAS/MAS v0.1	Pending
Integration	HAS/MAS v0.2	Pending

16.9 8. Performance Targets

16.9.1 8.1 Latency Targets

- **Descriptor Injection:** < 30 cycles (HIVE-C → RAPIDS)
- **Command Response:** < 1 µs @ 100 MHz
- **Monitoring Aggregation:** < 100 cycles for all 16 SERV status reads
- **Context Switch:** < 25 cycles (including quiesce)

16.9.2 8.2 Monitoring Overhead

- **SERV Monitor Overhead:** < 5% of compute time
- **Congestion Detection Latency:** < 10 cycles
- **Periodic Reporting Interval:** 1000 cycles (configurable)

See: `docs/hive_has/ch06_performance/01_performance.md` for the full target set and the tradeoff tables behind them.

16.10 9. Integration Guidelines

16.10.1 9.1 HIVE + RAPIDS + Delta Integration

```
// Top-level integration example (architectural port names)
module system_top (
    input  logic        clk_100mhz,
    input  logic        rst_n,
    // Host interface
    input  logic [31:0] host_cmd,
    output logic [31:0] host_resp,
    // External memory (for RAPIDS)
    // ... AXI4 memory interface ...
);

    // HIVE instance
    hive_top u_hive (
        .aclk           (clk_100mhz),
        .aresetn        (rst_n),
        .host_cmd       (host_cmd),
        .host_resp      (host_resp),
        .axis_cda_*     (hive_to_delta_cda),
        .axis_status_*  (delta_to_hive_status)
    );

    // Delta Network instance
    delta_network_4x4 u_delta (
        .aclk           (clk_100mhz),
        .aresetn        (rst_n),
        .tile16_*       (rapids_interface),  // RAPIDS virtual tile
        .tile17_*       (hive_interface)     // HIVE-C virtual tile
    );

    // RAPIDS instance
    rapids_top u_rapids (
        .aclk           (clk_100mhz),
        .aresetn        (rst_n),
        .axis_cda_*     (rapids_interface),  // From Delta
        .m_axi_*        (memory_interface)   // To external memory
    );

endmodule
```

See: `docs/hive_has/ch07_integration/01_integration_and_verification.md` for the full integration and verification strategy.

16.11 10. Development Status

16.11.1 10.1 Current Phase

Phase: Specification (HAS/MAS v0.2 complete, 2026-10-12)

- Hardware Architecture Specification: complete (7 chapters)
- Micro-Architecture Specification: complete (6 chapters)
- RTL: pending (roadmap in the HAS, Chapter 7)

See: nothing tracked – the vault area was deleted 2026-09-27 (unstarted); the former items are in git history.

16.11.2 10.2 Roadmap

Near-Term (Q4 2025): - HIVE-C RTL implementation (Phase 1, per HAS ch.7 roadmap) - SERV monitor wrapper implementation - Control network implementation

Long-Term (2026+): - Configuration manager implementation - HIVE-C firmware development - SERV firmware development - Integration with RAPIDS and Delta - FPGA deployment and testing

16.11.3 10.3 HIVE-C core strategy — VexRiscv now, falcon-flavored later

HIVE-C's core sits behind the `hive_c_loader` contract (AXIL loader window, load-then-run); the LDR testplan verifies that contract, not the core. That deliberate isolation makes the core **swappable by parameter** (`CORE_FLAVOR`, to be recorded in the MAS): `vexriscv` is the default today, `peregrine` is the intended successor when falcon rung 3 lands. The swap touches only the wrapper's core-facing glue (VexRiscv native ibus/dbus cmd/rsp vs peregrine's caches-behind-AXI4); the loader, firmware, and testplan carry over unchanged.

Two purposes, per the owner (2026-10-13):

1. **System proving ground for the falcon ladder.** Every falcon rung is verified in isolation (spike lockstep, riscv-formal, ISA batteries) but never *in a system*. HIVE-C is the vehicle: real control firmware, real interrupts, a real fabric — the LDR plan becomes peregrine's bring-up acceptance test the day its RTL exists.
2. **The "does a more powerful RV make a difference" experiment.** The PoC's headline latencies are fabric-bound (a stronger core will not move the 30-cycle injection target). The differences to measure: interrupt-driven firmware architecture (alert-to-action latency vs polling), scheduler worst case under load, and headroom at the 64-SERV expansion scale. Same firmware, same testplan, measure the deltas — if there is no system-level gain, that is a finding about HIVE's workload class, not a failed experiment.

16.12 11. Educational Value

HIVE demonstrates: - Hierarchical processor architecture (master + agents) - Distributed monitoring and control - RISC-V processor integration (VexRiscv, SERV) - Inband control packet injection - Dynamic network reconfiguration - Resource budget management - Control network design patterns

Target Audience: - Advanced RTL designers - Network-on-chip architects - Embedded systems engineers - RISC-V integration specialists

---

16.13 13. Documentation Generation

16.13.1 13.1 Generating PDF/DOCX from the Specifications

Tool: `bin/md_to_docx.py` (repo root) via the wrapper scripts:

```
cd projects/components/compute-eng-ip/hive-serv/docs
./generate_has_pdf.sh            # HIVE_HAS_v0.2.docx / .pdf
./generate_mas_pdf.sh            # HIVE_MAS_v0.2.docx / .pdf
# Optional revision override:
./generate_has_pdf.sh --rev 0.2
```

The scripts check dependencies (python-docx, PyYAML, pandoc, LibreOffice), then invoke `md_to_docx.py` with the book's index, styles yaml, title page, ToC, and page breaks. Built books are checked in alongside their sources.

Requirements: Python 3.6+, pandoc, LibreOffice (PDF export), and the repo's `bin/md_to_docx.py`.

16.14 14. References

16.14.1 14.1 Internal Documentation

- **HIVE HAS:** `docs/hive_has/` ← **Primary external-contract reference**
- **HIVE MAS:** `docs/hive_mas/` ← **Primary implementation-intent reference**
- **Master PRD:** `/PRD.md`
- **Repository Guide:** `/CLAUDE.md`

16.14.2 14.2 Related Subsystems

- **RAPIDS:** `projects/components/dma-ip/rapids/` - DMA engine controlled by HIVE-C
- **Delta Network:** `projects/components/fabric-gen-ip/delta/` - AXIS fabric generator (the HIVE fabric; owner: same fabric as the archived NoC draft)
- **AMBA/Ocelus:** `rtl/amba/ocelus/` - Monitor infrastructure for observability
- **Common:** `rtl/common/` - Building blocks

16.14.3 14.3 External References

- **VexRiscv:** https://github.com/SpinalHDL/VexRiscv
- **SERV:** https://github.com/olofk/serv
- **RISC-V ISA:** https://riscv.org/technical/specifications/
- **AXI4-Stream:** ARM IHI 0051A
- **HIVE Reference Library** (`docs/references.md`) — annotated bibliography (NoC/interconnect, network processors, RISC-V, SoC integration, modeling, verification, FPGA practice), compiled from the owner calibre library + spec-cited papers 2026-10-13

Document Version: 0.2 **Last Updated:** 2026-10-12 **Review Cycle:** Monthly during active development **Owner:** RTL Design Sherpa Project

---

16.15 Navigation

- **← Back to Root:** `/PRD.md`
- **Hardware Architecture Specification:** `docs/hive_has/hive_has_index.md`
- **Micro-Architecture Specification:** `docs/hive_mas/hive_mas_index.md`
- **Quick Start:** `README.md` (to be created)
- **AI Guidance:** `CLAUDE.md`
- **Tasks:** none tracked (vault area deleted 2026-09-27, unstarted)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

17 Claude Code Guide: HIVE Component

Version: 0.2 **Last Updated:** 2026-10-12 **Purpose:** AI-specific guidance for working with HIVE subsystem

---

17.1 Quick Context

What: HIVE (Hierarchical Intelligent Vector Environment) - Distributed control and monitoring subsystem for RAPIDS/Delta Network **Status:** HAS/MAS v0.2 published 2026-10-13; Phase 1 RTL in progress (descriptor_gen, axis_cda_master, ctrl_network CN-01..03 verified at gate/func; remaining blocks + the CN-04 rotation issue tracked in vault hive-serv TASK-000) **Your Role:** Help users understand hierarchical RISC-V architecture, distributed monitoring, and network reconfiguration

Primary Specifications: - HAS (`projects/components/compute-eng-ip/hive-serv/docs/hive_has/`) — external contract - MAS (`projects/components/compute-eng-ip/hive-serv/docs/hive_mas/`) — block-level implementation intent

---

17.2 Critical Rules for This Component

17.2.1 Rule #2: Always Reference the Specifications

This subsystem's technical documentation lives in: - `projects/components/compute-eng-ip/hive-serv/docs/hive_has/` — architecture, interfaces, packet formats, programming model - `projects/components/compute-eng-ip/hive-serv/docs/hive_mas/` — module trees, clocking/reset, cycle-level behavior, verification hooks

Before answering technical questions:

```
ls projects/components/compute-eng-ip/hive-serv/docs/hive_has/
ls projects/components/compute-eng-ip/hive-serv/docs/hive_mas/
cat
projects/components/compute-eng-ip/hive-serv/docs/hive_has/hive_has_in
dex.md
```

Your answer should: 1. Provide direct answer/code 2. **Then link to the spec:** "See `projects/components/compute-eng-ip/hive-serv/docs/hive_has/{file}.md` (or `hive_mas/`) for the full specification"

17.2.2 Rule #3: HIVE System Context

IMPORTANT: HIVE is part of the larger RAPIDS/Delta/HIVE ecosystem:

1. **RAPIDS** - DMA engine with inband descriptor injection
   - Virtual Tile 16 in Delta Network
   - Controlled by HIVE-C via CDA packets
2. **Delta Network** - 4×4 mesh Network-on-Chip (`projects/components/fabric-gen-ip/delta/`)
   - 16 physical compute tiles (0-15)
   - 2 virtual tiles (16=RAPIDS, 17=HIVE-C)
   - Four packet types (PKT_DATA, CDA, PKT_CONFIG, PKT_STATUS)
3. **HIVE** - Distributed control and monitoring
   - HIVE-C master controller (VexRiscv) - Virtual Tile 17
   - 16 SERV monitors (one per physical tile)
   - Control network for HIVE-C ↔️ SERV communication
   - Configuration manager for network reconfiguration

When user mentions HIVE: Clarify whether they're asking about the entire ecosystem or just the HIVE control subsystem.

17.3 Architecture Quick Reference

17.3.1 Block Organization (HIVE System)

```
HIVE System Architecture
├── Block 1: HIVE-C Master Controller (VexRiscv)
│   ├── VexRiscv "Small" RV32IM core
│   ├── 32KB instruction memory
│   ├── 32KB data memory
│   ├── Descriptor generation (CDA packets)
│   ├── Control network master
│   └── AXIS master (to Delta Network)
│
├── Block 2: SERV Monitors (16 instances)
│   ├── SERV bit-serial RISC-V (RV32I)
│   ├── 2KB instruction + 2KB data memory each
│   ├── Traffic counters (pkt_rx/tx_count)
│   ├── Buffer occupancy monitors
│   ├── Congestion detection logic
│   └── AXIS injector (PKT_CONFIG, PKT_STATUS)
│
├── Block 3: Control Network
│   ├── Star topology (HIVE-C ↔ 16 SERVs)
│   ├── Round-robin arbiter (SERV → HIVE-C)
│   ├── Command delivery (HIVE-C → SERV)
│   └── Status aggregation (SERV → HIVE-C)
│
└── Block 4: Configuration Manager
    ├── 4 virtual configuration contexts
    ├── Context 0: Systolic Mode
    ├── Context 1: Packet-Switched Mesh (XY routing)
    ├── Context 2: Tree Reduction
    ├── Context 3: Custom/Debug
    └── Atomic topology switching protocol
```

See: - `docs/hive_has/ch03_architecture/01_subsystem_architecture.md` (block specs) - `docs/hive_mas/ch02_blocks/` (module trees)

17.4 Common User Questions and Responses

17.4.1 Q: "What's the difference between HIVE-C and SERV monitors?"

A: Hierarchical control architecture:

Aspect	HIVE-C Controller	SERV Monitors (16×)
Core	VexRiscv RV32IM	SERV RV32I (bit-serial)
Resources	1,400 LUTs, 8 BRAM	125 LUTs, 1 BRAM each
Role	Global coordination	Per-tile monitoring
Responsibilities	Descriptor scheduling, network reconfiguration, performance aggregation	Traffic monitoring, congestion detection, local packet injection
Network Position	Virtual Tile 17	Co-located with physical tiles 0-15

Key Point: HIVE-C is the master controller, SERV monitors are lightweight per-tile agents.

See: - `docs/hive_has/ch03_architecture/01_subsystem_architecture.md` - `docs/hive_mas/ch02_blocks/02_hive_c_controller.md` and `03_serv_monitor.md`

17.4.2 Q: "How does HIVE-C send descriptors to RAPIDS?"

A: Inband CDA packet injection via Delta Network:

```
1. HIVE-C generates 256-bit CDA descriptor
   ↓
2. AXIS master transmits to Delta Network
   - TUSER = 2'b01 (CDA packet type)
   - TDEST = 16 (RAPIDS virtual tile)
   ↓
3. Delta Network routes to RAPIDS (virtual tile 16)
   ↓
4. RAPIDS receives CDA packet
   - Descriptor engine processes
   - Scheduler activates appropriate data path
```

Advantages over memory-mapped descriptors: - Lower latency (~10-20 cycles vs. 100+) - No memory bus contention - Inband delivery through existing network

See: - `docs/hive_has/ch04_interfaces/02_packet_formats.md` (descriptor format) - `docs/hive_mas/ch04_behavior/01_descriptor_injection_cycle.md` (cycle-by-cycle)

17.4.3 Q: "How does network reconfiguration work?"

A: Atomic context switching via Configuration Manager:

```
Switching Sequence:
1. HIVE-C issues CONFIG_PREPARE command
   - Target tiles flush in-flight packets
   - Routers enter quiescent state (5-10 cycles)

2. HIVE-C broadcasts SET_ROUTING_MODE (PKT_CONFIG packet)
   - TUSER = 2'b10 (PKT_CONFIG type)
   - All tiles simultaneously update mode register
   - Context switch occurs in single cycle

3. HIVE-C issues CONFIG_ACTIVATE command
   - Tiles resume operation with new routing
   - Total latency: 10-20 cycles (deterministic)
```

Virtual Configuration Contexts: - Context 0: Systolic Mode (nearest-neighbor) - Context 1: Packet-Switched Mesh (XY routing) - Context 2: Tree Reduction (hierarchical aggregation) - Context 3: Custom/Debug (user-programmable)

See: - `docs/hive_has/ch05_programming_model/04_network_reconfiguration.md` (workflow) - `docs/hive_mas/ch04_behavior/03_context_switch_sequence.md` (FSM/cycles)

17.4.4 Q: "What do SERV monitors track?"

A: Per-tile traffic and congestion metrics:

Counters (32-bit, wrapping): - `pkt_rx_count[4]` - Received packets per direction (N/S/E/W) - `pkt_tx_count[4]` - Transmitted packets per direction - `pkt_local_inject` - Packets injected by local compute element - `pkt_local_consume` - Packets consumed by local compute element - `buffer_overflow_count` - FIFO overflow events - `stall_cycles` - Cycles with valid packet waiting for credits

Real-Time Metrics: - `buffer_occupancy[4]` - Current FIFO fill levels (8-bit per direction) - `congestion_flag` - Boolean indicating occupancy > threshold - `error_flags` - Parity errors, protocol violations

Reporting: - Periodic status to HIVE-C (every 1000 cycles, configurable) - Immediate alerts on congestion/errors via PKT_STATUS packets

See: - `docs/hive_has/ch03_architecture/01_subsystem_architecture.md` (Block 2) - `docs/hive_mas/ch02_blocks/03_serv_monitor.md` (counter/tap design)

17.5 Key Documentation Links

17.5.1 Primary Technical Specifications

- `docs/hive_has/hive_has_index.md` - Hardware Architecture Specification index
- `docs/hive_mas/hive_mas_index.md` - Micro-Architecture Specification index
- `docs/hive_has/ch04_interfaces/` - Interfaces, packet formats, register maps
- `docs/hive_mas/ch04_behavior/` - Cycle-level behavior of key operations

17.5.2 Related Component Specifications

- `projects/components/dma-ip/rapids/docs/rapids_beats_has/` and `rapids_beats_mas/` - RAPIDS DMA specification (Beats architecture)
- `projects/components/fabric-gen-ip/delta/docs/delta_has/` and `delta_mas/` - Delta Network specification

17.6 Documentation Generation

17.6.1 Generating PDF/DOCX from the Specifications

Tool: `bin/md_to_docx.py` (repo root) via the wrapper scripts:

```
cd projects/components/compute-eng-ip/hive-serv/docs
./generate_has_pdf.sh            # HIVE_HAS_v0.2.docx / .pdf
./generate_mas_pdf.sh            # HIVE_MAS_v0.2.docx / .pdf
```

The scripts check dependencies (python-docx, PyYAML, pandoc, LibreOffice), then invoke `md_to_docx.py` with the book's index, styles yaml, title page, ToC, and page breaks. Built books are checked in alongside their sources.

Requirements: Python 3.6+, pandoc, LibreOffice (PDF export), and the repo's `bin/md_to_docx.py`.

Debug mode (merged markdown inspection):

```
python bin/md_to_docx.py \

projects/components/compute-eng-ip/hive-serv/docs/hive_has/hive_has_in
dex.md \
    -o output.docx --debug-md
```

17.7 Remember

1. **Link to the specs** - `docs/hive_has/` (architecture) and `docs/hive_mas/` (implementation intent)
2. **Hierarchical architecture** - HIVE-C (master) + 16 SERV monitors (agents)
3. **Virtual tiles** - RAPIDS=16, HIVE-C=17 in Delta Network topology
4. **Four packet types** - PKT_DATA, CDA, PKT_CONFIG, PKT_STATUS (TUSER[1:0])
5. **Configuration contexts** - 4 virtual routing modes with atomic switching
6. **Control network** - Separate star topology for HIVE-C ↔️ SERV communication
7. **Monitoring** - Per-tile traffic stats, congestion detection, error reporting
8. **Pre-RTL** - everything is specification; the MAS marks open items explicitly

---

Version: 0.2 **Last Updated:** 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

18 RAPIDS Beats Architecture HAS

Version: 0.15 **Date:** 2026-10-20 **Purpose:** Hardware Architecture Specification for RAPIDS Phase 1 “Beats” Architecture **Classification:** External Interface Specification

---

18.1 Document Organization

Note: All chapters linked below for automated document generation.

18.1.1 Front Matter

- Document Information
- Revision History
- Acronyms and Definitions

18.1.2 Chapter 1: Overview

- Product Overview
- Key Features
- System Context

18.1.3 Chapter 2: Architecture

- Block Diagram
- Data Flow
- Channel Architecture

18.1.4 Chapter 3: Interfaces

- Interface Summary
- AXI4 Master Interface
- AXIS Interface
- APB Configuration Interface
- Monitor Bus Interface
- CXS Network Interface

18.1.5 Chapter 4: Use Cases

- Network-to-Memory Transfer
- Memory-to-Network Transfer
- Descriptor Chaining
- Multi-Channel Operation

18.1.6 Chapter 5: Programming Model

- Descriptor Format
- Register Map
- Initialization Sequence

- Error Handling

18.1.7 Chapter 6: Performance

- Throughput Targets
- Latency Characteristics
- Resource Estimates

18.2 Quick Reference

18.2.1 RAPIDS Beats at a Glance

RAPIDS Beats Feature Summary

Feature	Specification
Architecture	Network-to-Memory Accelerator
Channels	8 independent DMA channels
Data Width	512-bit (configurable)
Address Width	64-bit
Descriptor Size	256-bit
AXI Protocol	AXI4 (Read/Write Masters)
Network Protocol	AXI-Stream (Sink/Source)
Monitoring	64-bit MonBus packets

18.2.2 Interface Summary

External Interface Summary

Interface	Type	Direction	Width	Purpose
Descriptor AXI	AXI4 Master	Bidirectional	256-bit	Descriptor fetch
Sink AXI	AXI4 Master	Write	512-bit	Memory write
Source AXI	AXI4 Master	Read	512-bit	Memory read
Sink AXIS	AXIS Slave	Input	512-bit	Network ingress
Source AXIS	AXIS Master	Output	512-bit	Network egress
MonBus	Custom	Output	64-bit	Monitoring

18.3 Related Documentation

- **RAPIDS Beats MAS** - Module Architecture Specification
- **PRD.md** - Product requirements
- **ADDRESS_INCREMENT_PATTERNS.md** - DMA addressing patterns

18.4 Specification Conventions

18.4.1 Signal Naming

- **Clock:** `aclk` (AXI clock domain)
- **Reset:** `aresetn` (active-low, async assert, sync deassert)
- **AXI Signals:** Standard ARM AMBA naming
- **AXIS Signals:** Standard AXI-Stream naming

18.4.2 Diagram Conventions

- **Mermaid:** Block diagrams and flowcharts
- **WaveDrom:** Timing diagrams
- **Tables:** Interface specifications and register maps

Last Updated: 2026-07-02 **Maintained By:** RAPIDS Architecture Team

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

19 Delta Hardware Architecture Specification Index

19.1 Overview

Version: 1.3 **Date:** 2026-10-20 **Purpose:** High-level hardware architecture specification for the Delta AXI4-Stream crossbar generator, including the planned mesh network mode

---

19.2 Related Modules

- **Delta MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide
- **QUICK_START.md** - Hands-on generator guide

19.3 Navigation

Note: Every chapter below is one source file; the document build assembles the spec from these links.

19.3.1 Front Matter

- Document Information

19.3.2 Chapter 1: Introduction

- Purpose and Scope
- Document Conventions
- Definitions and Acronyms

19.3.3 Chapter 2: System Overview

- Use Cases
- Key Features
- System Context

19.3.4 Chapter 3: Architecture

- Block Diagram
- Data Flow
- Protocol Support

19.3.5 Chapter 4: Interfaces

- AXI-Stream Base Interface
- AXI-Stream Rich Interface (TSTRB + AMBA5)
- Clock and Reset

19.3.6 Chapter 5: Performance

- Throughput Characteristics
- Latency Analysis
- Resources

19.3.7 Chapter 6: Integration

- System Requirements
- Parameter Configuration
- Verification Strategy
- Synthesis and Implementation

19.3.8 Chapter 7: Mesh Network

- Hierarchy
- Cell and Flit
- Routing and Configuration
- Flow Control and Deadlock
- Quality of Service
- Multicast
- Systolic Support and Fabric Modes
- Observability and DFT
- Performance and Verification
- Deferred Items
- Resilient Routing: Open Hazards

Last Updated: 2026-10-14 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

20 RISC-V Falcon Suite

Five independent, clean-slate RISC-V cores whose primary product is **education**, in the same spirit as the pumice/scoria/andesite memory-controller line. Each rung is a complete, verified, documented design small enough to hold in your head; the rungs ascend in microarchitectural complexity so the *diff between neighbors is the lesson*. Rungs do not share RTL — each core is the cleanest possible example of its level, not the cheapest to produce. Educational benefit over IP; no rung chases performance or benchmarks.

Design spec (source of truth for scope, ISA, verification, packaging): `docs/superpowers/specs/2026-10-06-riscv-falcon-suite-design.md`.

20.1 The ladder

Each rung carries a raptor-codename, real falcons first (species in strict size order, tracking complexity). The falcon run ends at gyrfalcon — the largest living species — so the top rung steps deliberately to mythology: garuda, the giant divine raptor, king of birds, one past the largest falcon. The codename is the RTL identifier prefix and the module/package prefix; the DIRECTORY is the compound `<codename>-<isa>` form — `kestrel-rv32i`, not `kestrel`. The bare codename is how the core is referred to in prose ("kestrel sees the whole machine in one cycle").

Directory	Codename	ISA	Microarchitecture	Status
`kestrel-rv32i/`	**kestrel**	RV32I	single-cycle	RTL complete; rv32ui battery 42/42 with spike lockstep; riscv-formal 42/42; doc book v0.1 published; board loader sim-verified
`merlin-rv32i/`	**merlin**	RV32I	5-stage in-order pipeline	Planned; structure only
`peregrine-rv32im/`	**peregrine**	RV32IM	advanced in-order: predictor, traps, L1 caches, AXI master	Planned; structure only
`gyrfalcon-rv32ima/`	**gyrfalcon**	RV32IM baseline; gyrfalcon-A flavor RV32IMA	out-of-order capstone: rename, ROB, reservation stations; two flavors — baseline and gyrfalcon-A (`A_EXT=1`:	Planned; structure only

Directory	Codename	ISA	Microarchitecture	Status
			LR/SC + AMO atomics study)	
`garuda-rv32imaf/`	**garuda**	RV32IMAF	gyrfalcon's OoO plus an FPU: FP register file + rename, fcsr precise flags, non-pipelined long-latency units	Planned; structure only (penciled in; beyond the 2026-10-06 design spec, which pins four rungs)

20.1.1 Moved: hive-serv

hive-serv — the SERV-based control/monitoring system (1 VexRiscv control plane + 16 SERV monitor cores) — left this area 2026-10-13 for `compute-eng-ip/hive-serv`: it *uses* RISC-V cores rather than teaching one, and it is the control half of the compute family (its other half is the cell CGRA), not a falcon-ladder rung. This directory is the ladder again, nothing more.

20.1.2 Tetrad engine: gatling-rv32itetrad

gatling-rv32itetrad/ is not a ladder rung either — it is the compute family's tetrad engine (owner PRD 2026-10-17): a barrel RV32I core with the Xcell extension (CFG/POP/PUSH/ECTX, custom-0), one engine per tetrad with four contexts — the four cells — sharing one F/D/X/W datapath. It replaces the cell PRD's ALU + execution-FSM block ("the program is the config") and answers the kestrel-for-the-compute-FSM question in kestrel's own lineage: same RV32I contract and lockstep flow, inverted pipeline lesson — N contexts hide one shared datapath's latency instead of stages hiding latency for one stream. Open decisions and its interface to the bf16 alu-infer block are tracked in vault TASK-001.

20.2 Goals per rung

kestrel — see everything. Single-cycle RV32I: PC, register file, ALU, immediate generator, control as an explicit decode truth table, a plainly documented combinational-read memory contract. The hovering falcon: the whole machine is visible in one cycle — including its cost, since the critical path *is* the machine, which is exactly the motivation for merlin. **Delivers:** the ISA (R/I/S/B/U/J formats, the full base integer set), the datapath, honest control logic.

merlin — flow it. The classic IF/ID/EX/MEM/WB pipeline on the deliberately identical RV32I ISA, so students diff *behavior*, not ISA. Forwarding from EX/MEM and MEM/WB with explicit priority, load-use interlock stall, branches resolved in EX predict-not-taken with flush. **Delivers:** pipeline registers, RAW hazards, forwarding priority, the load-use bubble, control-hazard flushes, why branch resolution stage matters.

peregrine — speed it up. Advanced in-order RV32IM: iterative multicycle multiply/divide (the structural-hazard lesson), BTB + 2-bit bimodal prediction with mispredict recovery, machine-mode CSRs with precise exceptions and CLINT-shaped interrupts, split blocking write-back L1 caches behind an AXI4 master on the repo fabric with MonBus visibility. **Delivers:** speculation and recovery, precise exceptions, the memory hierarchy, bus protocol — and ties the whole repo together (memory controllers, cache IP line, AMBA fabric).

gyrfalcon — hide everything. Out-of-order capstone in two flavors (added 2026-10-11): the baseline — peregrine's RV32IM exactly — and gyrfalcon-A (`A_EXT=1`): LR/SC reservations, store-conditional failure, and AMOs, the atomicity study — so every peregrine battery still carries in both. Dual-issue dispatch, explicit rename map + free list, 32-entry ROB with in-order commit, three reservation stations, wakeup/select over common data buses, speculative dispatch with checkpoint-restore rename rollback. **Delivers:** register renaming and WAR/WAW elimination, the ROB as an undo log, out-of-order completion with in-order retirement — the honest, complete OOO story — and the atomicity study: reservations, SC failure, AMOs, acquire/release as memory-model operations.

garuda — float it. The RV32IMAF delta on gyrfalcon's OoO: a 32-entry FP register file with its own rename, an `fcsr` whose five accrued exception flags (NV/DZ/OF/UF/NX) stay precise-exception-clean through the ROB, and the repo's ieee754 fp32 arithmetic behind the reservation stations — multi-cycle FMUL/FADD and the non-pipelined Goldschmidt FDIV / Newton-rsqrt FSQRT (9–11 cycle class) as the long-latency-unit lesson. Rounding hard-wired RNE; any other requested mode raises NV, keeping the CSR story honest without a dynamic rounding-mode datapath. **Delivers:** long-latency execution scheduling, FP architectural state, and the F-extension ISA delta (F regs, `fcsr`, FMV/FCVT/FSGNJ/FCLASS) — consuming the math library's `math_ieee754_2008_fp32_*` blocks end-to-end.

20.3 Packaging (per rung, the MC-line treatment)

- **Doc book** — *Simplified RV32I: kestrel* and successors, Memory Notes branding, built through `bin/md_to_docx.py` to DOCX/PDF, following the Simplified DFI books' precedent. Each book lives in its rung's `docs/`; the kestrel edition is published at `kestrel-rv32i/docs/simplified_rv32i/`.
- **Formal** — riscv-formal-style ISA compliance where it pays: full on kestrel, retire-interface on merlin, targeted properties (exception precision, recovery, in-order commit, deadlock freedom) on peregrine/gyrfalcon.
- **Golden model** — spike ISS lockstep in the rds-dv cocotb framework; the classic `rv32ui`/`rv32um` riscv-tests batteries as the compliance gate.
- **Drill app** — `bin/apps/core_drills` (ddr_drills architecture): pipeline stage-viewer with forwarding arrows, hazard quiz, branch-predictor drill, and a rename/ROB viewer. App design gets its own doc once merlin's RTL is fixed.

20.4 References and layout

Common primary sources (the RISC-V ISA manuals) live in `references/` and are cited by chapter, not page.

Per-rung skeleton follows the dma-ip/stream component layout convention: `rtl/`, `docs/`, `dv/` (cocotb + lockstep testbenches). Shared suite doctrine lands in `docs/` as it is written. Per-rung `PRD.md`/`CLAUDE.md` arrive with each rung's implementation plan.